

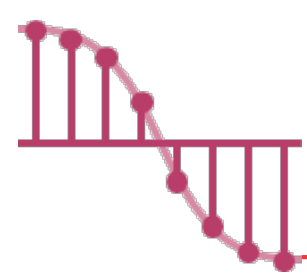
Chap 6 | **Graphs**

海大資訊工程系
蔡東佐



Outline

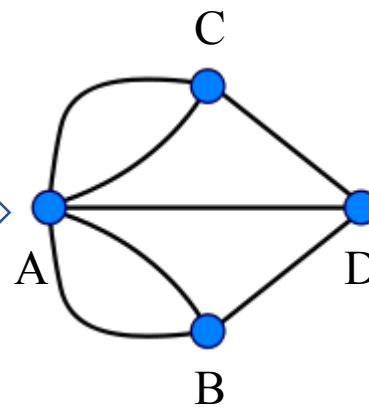
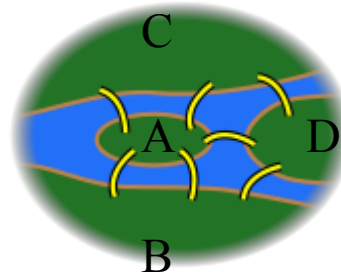
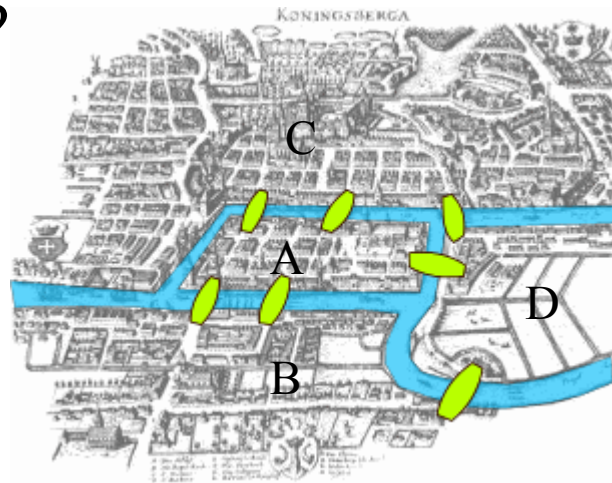
- The Graph Abstract Data Type (圖形抽象資料型態)
- Elementary Graph Operations (基本的圖形運算)
- Minimum Cost Spanning Trees (最小成本生成樹)
- Shortest Paths (最短路徑)



Königsberg Bridge Problem

七橋問題

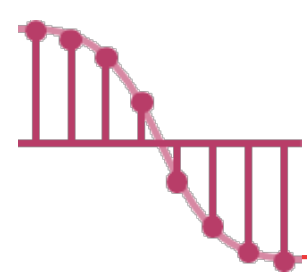
➤ Can we walk across all the bridges **exactly once** in returning back to the starting land area ?



➤ Transferring to "graph" model

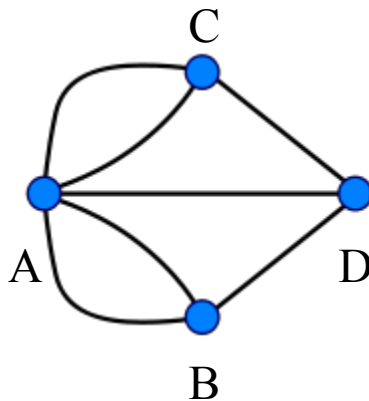
✓ Land → vertex

✓ Bridge → edge

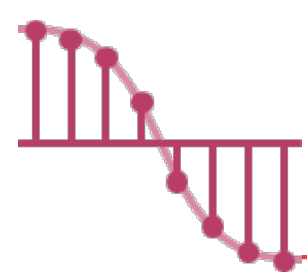


Eulerian Walk

- A walk that does this is called **Eulerian**.
- Euler showed that the graph contains an Eulerian walk **if and only if (iff) the degree of each vertex is even**.
- He answered the Königsberg problem in the negative.
- There is no Eulerian walk for Königsberg problem as all four vertices are of odd degree



degree (分支度): 進入頂點的邊數



Definition of A graph _{1/2}

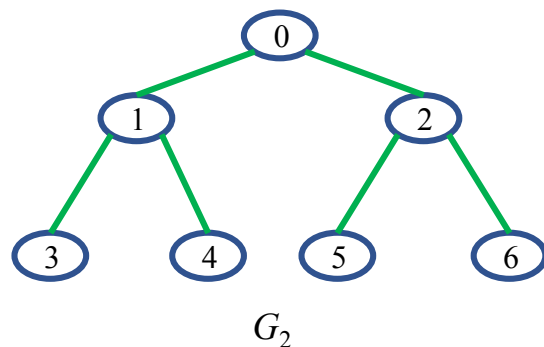
➤ A **graph**, G , consists of two sets, V and E

✓ V is a finite, nonempty set of vertices. 点是有限的

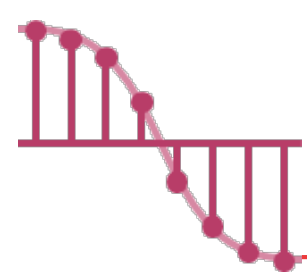
✓ E is a set of pairs of vertices; these pairs are called edges.

✓ $V(G)$ and $E(G)$ will present the sets of vertices and edges, respectively, of G .

✓ We will also write $G = (V, E)$ to represent a graph.



$V(G_2) = \{0, 1, 2, 3, 4, 5, 6\}$
 $E(G_2) = \{(0, 1), (0, 2), (1, 3), (1, 4), (2, 5), (2, 6)\}$



Definition of A graph _{2/2}

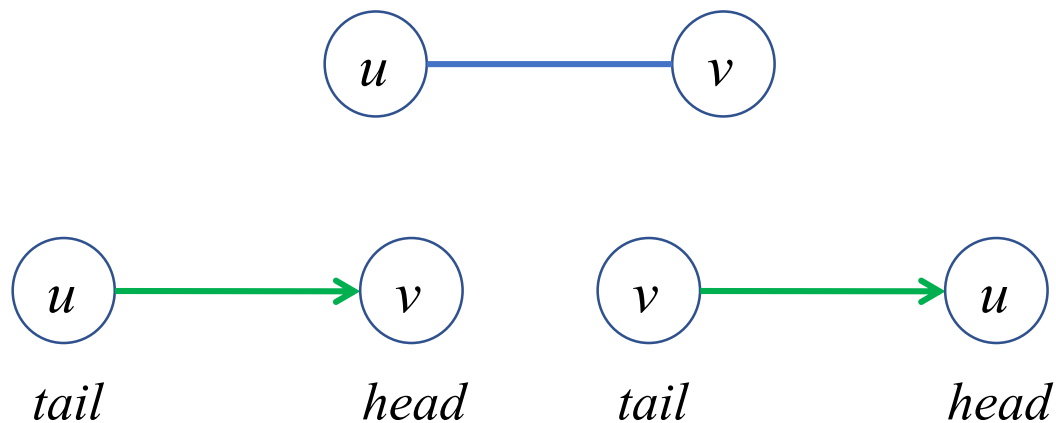
➤ Graphs have two different types:

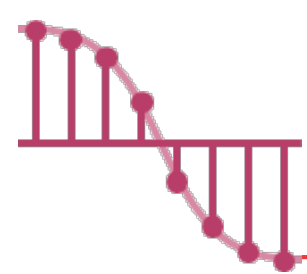
無向圖

✓ **Undirected graph**: (u, v) and (v, u) represent the same edge

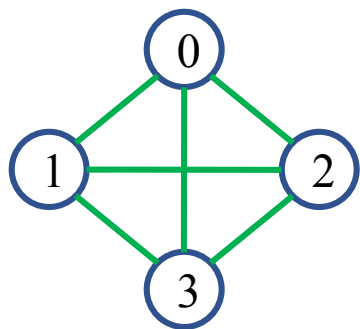
有向圖

✓ **Directed graph**: a directed pair $\langle u, v \rangle$ has u as the tail and the v as the head. So $\langle u, v \rangle$ and $\langle v, u \rangle$ indicate different edges



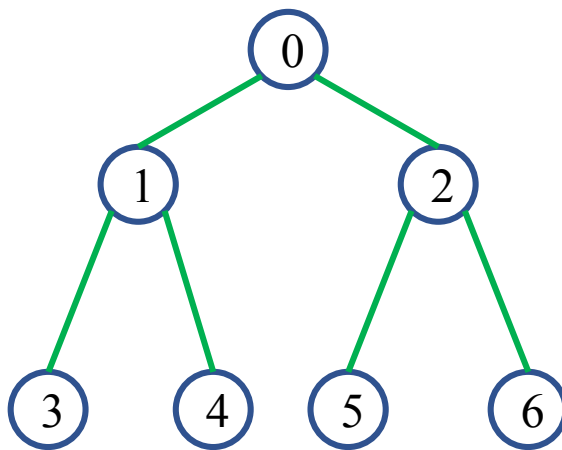


Three Sample Graphs



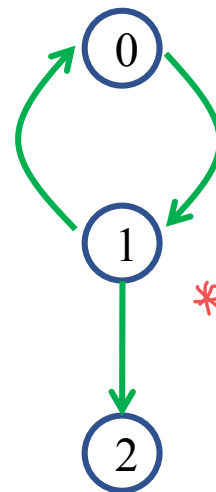
G_1

$V(G_1) = \{0, 1, 2, 3\}$
 $E(G_1) = \{(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)\}$



G_2

$V(G_2) = \{0, 1, 2, 3, 4, 5, 6\}$
 $E(G_2) = \{(0, 1), (0, 2), (1, 3), (1, 4), (2, 5), (2, 6)\}$



* Directed Graph

G_3

$V(G_3) = \{0, 1, 2\}$
 $E(G_3) = \{<0, 1>, <1, 0>, <1, 2>\}$

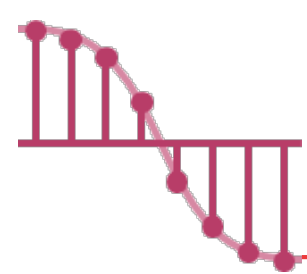
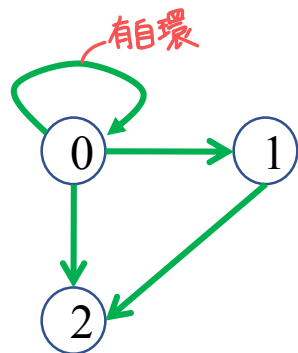


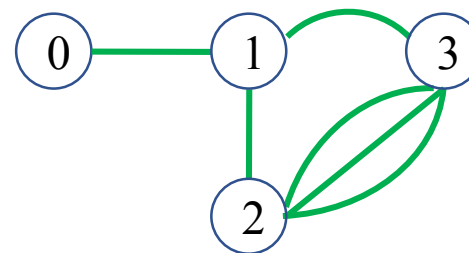
Figure 6.3: Examples of graphlike structures

➤ We impose the following restrictions on graphs:

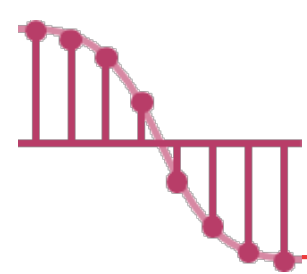
- ✓ A graph may not have an edge from a vertex, v , back to itself. That is, **edges of the form (v, v) and $\langle v, v \rangle$ are not legal**. Such edges are known as **self edges** or **self loops**.
- ✓ A graph may not have multiple occurrences of the same edges. Such graphs are known as **multigraph**.



Graph with a self edge

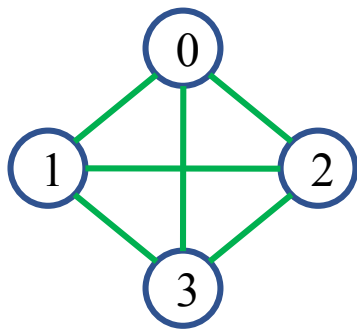


Multigraph



Complete Graph (完全圖)

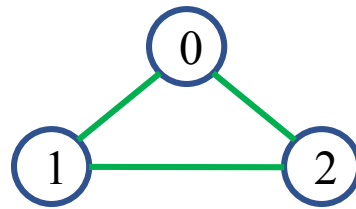
- The number of distinct unordered pairs (u, v) with $u \neq v$ in a graph with n vertices: $n(n-1)/2$.
- An n -vertex, **undirected graph** with exactly $n(n-1)/2$ edges is said to be **complete**.
- In the case of a **directed graph** on n vertices, the maximum number of edges is $n(n-1)$.



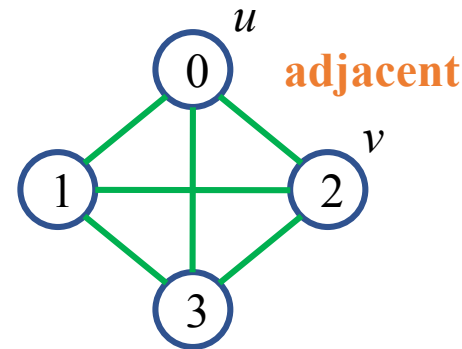
A complete graph with 4 vertices and 6 edges.

Subgraph

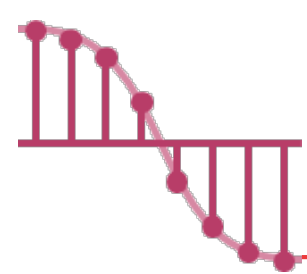
- If (u, v) is an edge in $E(G)$, then the vertices u and v are **adjacent** and that the edge (u, v) is **incident** on vertices u and v .
相鄰
接附
- A **subgraph** (子圖) of G is a graph G' such that $V(G') \subseteq V(G)$ and $E(G') \subseteq E(G)$.



G'

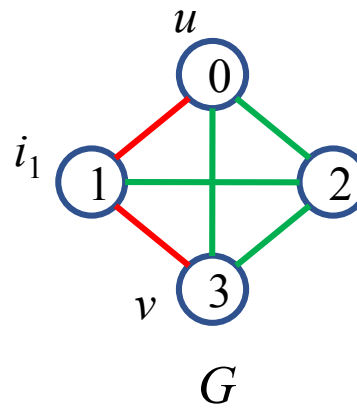


G



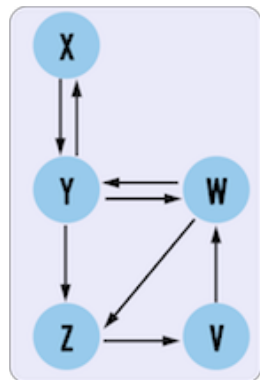
Path

- A **path**(路徑) from vertex u to vertex v in graph G is a sequence of vertices $u, i_1, i_2, \dots, i_k, v$, such that $(u, i_1), (i_1, i_2), \dots, (i_k, v)$ are edges in $E(G)$.
- If G is **directed**, the path consists of $\langle u, i_1 \rangle, \langle i_1, i_2 \rangle, \dots, \langle i_k, v \rangle$ edges in $E(G)$.

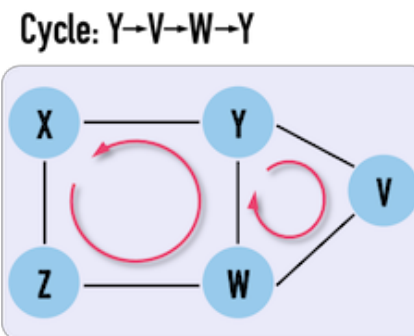


Path and Cycle

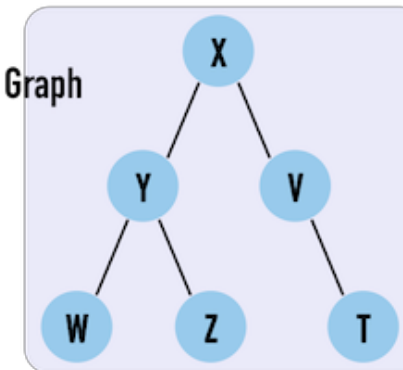
- The **length of a path** is **the number of edges on it**.
- A **simple path** a path in which all vertices, except possibly the first and the last, are distinct.
- A **cycle** is a **simple path** in which **the first and last vertices are the same**.
- A **tree** is an **acyclic** graph.

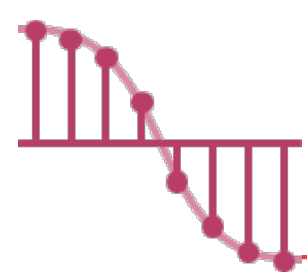


Graph G
Path : $X \rightarrow Y \rightarrow Z \rightarrow V \rightarrow W \rightarrow Y \rightarrow X$
Length : 6
Simple Path: $X \rightarrow Y \rightarrow Z$,
Not Simple Path: $Y \rightarrow X \rightarrow Y \rightarrow W$

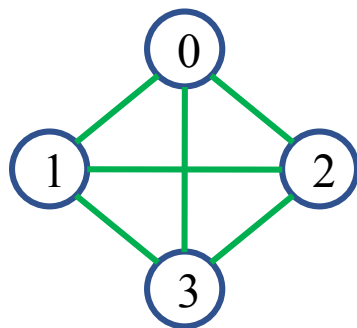


Tree:
Acyclic Graph

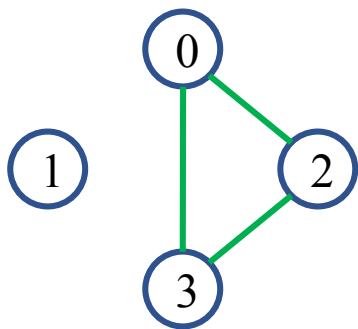




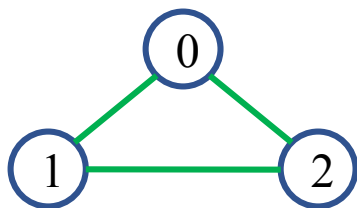
Some subgraphs



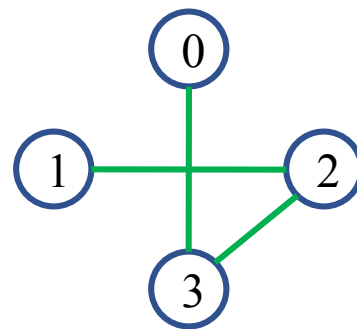
G_1



(a)

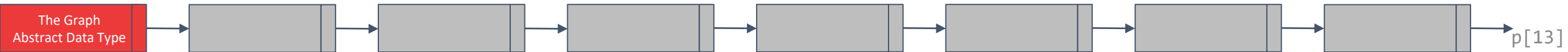


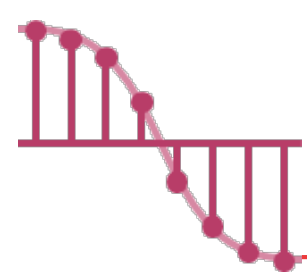
(b)



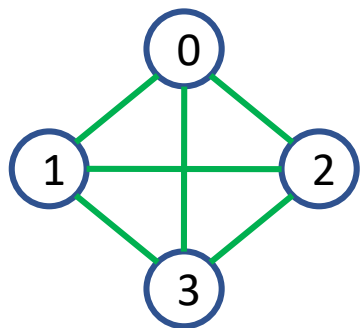
(c)

Graphs (a), (b) and (c) are the subgraph of G_1





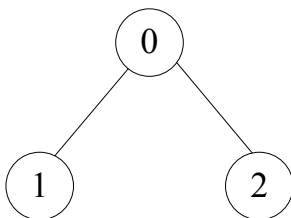
Some subgraphs



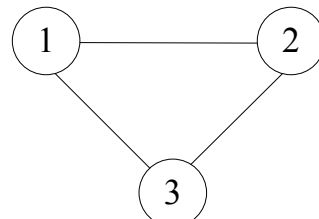
G_1



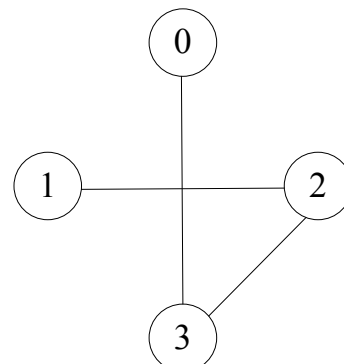
(i)



(ii)

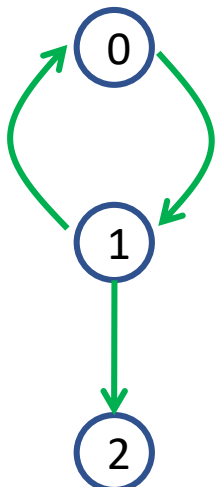


(iii)



(iv)

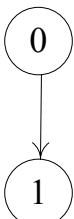
(a) 一些 G_1 的子圖



G_3



(i)



(ii)

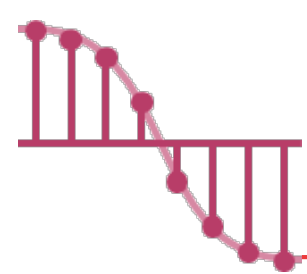


(iii)



(iv)

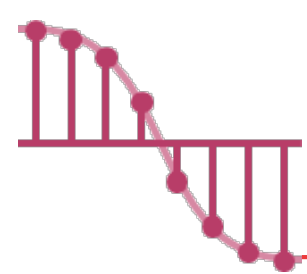
(b) 一些 G_3 的子圖



Connected

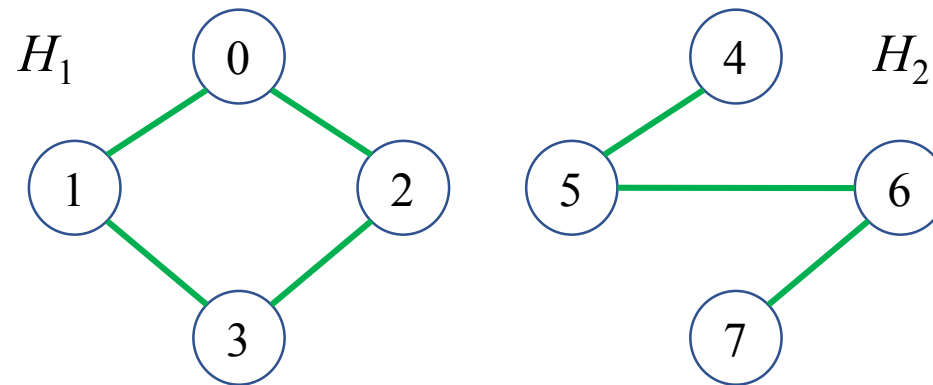
- In an undirected graph, G , two vertices u and v are **connected** **iff** there is a path in G from u to v .





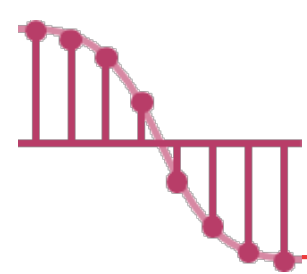
Connected Component

- An undirected **graph** is **connected iff** for every pair of distinct vertices u and v in $V(G)$ there is a path from u to v in G .
- A **connected component** (or simply a component), H , of an undirected graph is a **maximal** connected subgraph.



G_4

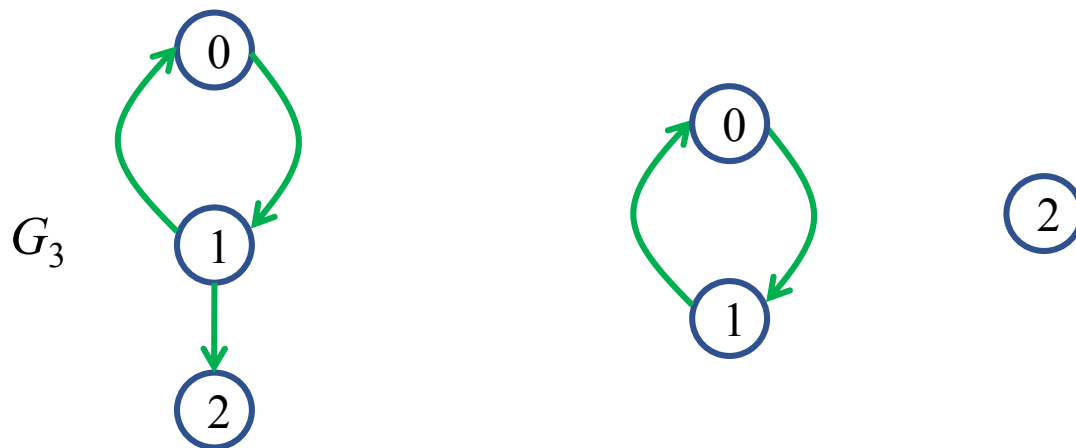
A Graph with two Connected Components



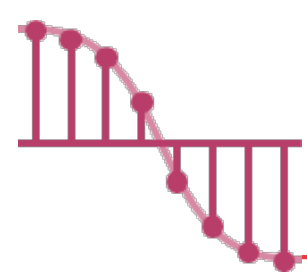
Strongly Connected Graph

有向图、兩張都有向可抵 → 強連通

- A **directed graph** G is said to be **strongly connected** iff for every pair of distinct vertices u and v in $V(G)$, there is directed path from u to v and also from v to u .
- A **strongly connected component** is a maximal subgraph that is strongly connected.

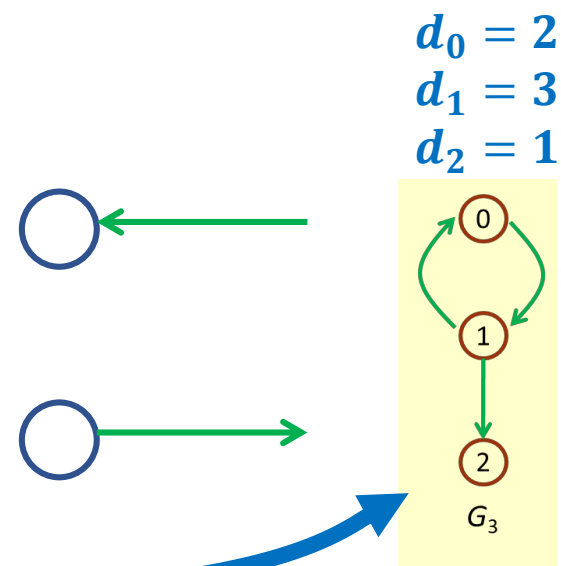


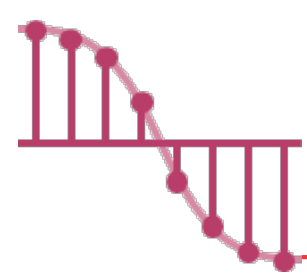
Strongly Connected Components of G_3



Degree of a Vertex

- The **degree** of a vertex is the number of edges incident to that vertex.
- If G is a directed graph:
 - ✓ The **in-degree** of a vertex is the number of edges for which vertex is head.
 - ✓ And, the **out-degree** of a vertex is the number of edges for which the vertex is the tail.
- If d_i is the degree of the vertex i in a graph with n vertices, then the number of edges is $(\sum_{i=0}^{n-1} d_i)/2$.





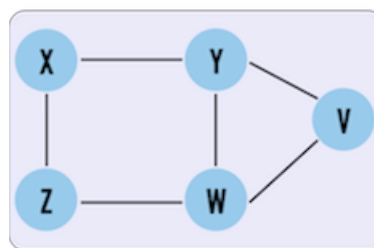
Graph Representations

➤ There are two most commonly used representations for graph:

✓ **Adjacency matrices** 鄰接矩陣

✓ **Adjacency lists** 鄰接列表

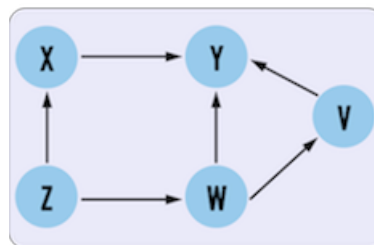
Undirected Graph



Adjacency Matrix

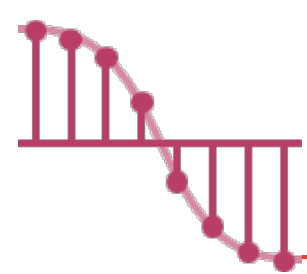
	X	Y	Z	W	V
X	0	1	1	0	0
Y	1	0	0	1	1
Z	1	0	0	1	0
W	0	1	1	0	1
V	0	1	0	1	0

Directed Graph



Adjacency Matrix

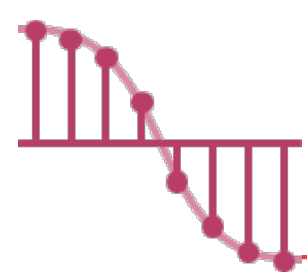
	X	Y	Z	W	V
X	0	1	0	0	0
Y	0	0	0	0	0
Z	1	0	0	1	0
W	0	1	0	0	1
V	0	1	0	0	0



Graph Representations

- There are two most commonly used representations for graph:
 - ✓ **Adjacency matrices**
 - ✓ **Adjacency lists**
- The choice of a particular representation will depend upon the application one has in mind and the functions one expects to perform on the graph.





Adjacency Matrix (相鄰矩陣)

- The **adjacency matrix** of G is a two-dimensional $n \times n$ array a , with the property that
 - ✓ $a[i][j] = 1$ iff the edge (i, j) is in $E(G)$
 - ✓ $a[i][j] = 0$ iff there is no such edge in G
 - ✓ The adjacency matrix for an undirected graph is symmetric.
- For an **undirected graph** the **degree** of any vertex i is its row sum.
- For an **directed graph** the row sum is its **out-degree** and the column sum is its **in-degree**.

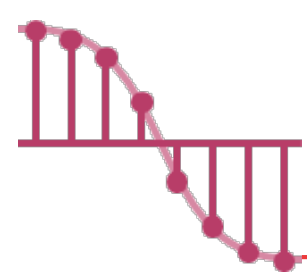
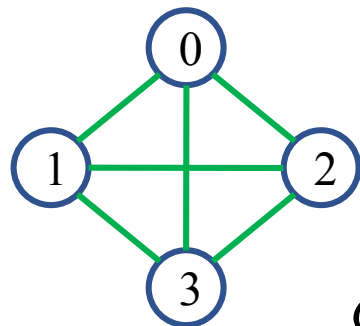


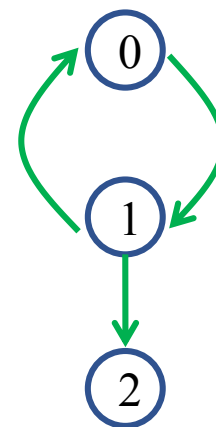
Figure 6.7: Adjacency matrices ^{1/2}



G_1

	0	1	2	3
0	0	1	1	1
1	1	0	1	1
2	1	1	0	1
3	1	1	1	0

The adjacency matrix of G_1



G_3

	0	1	2
0	0	1	0
1	1	0	1
2	0	0	0

The adjacency matrix of G_3

The adjacency matrix for an undirected graph is symmetric. (無方向圖形的相鄰矩陣是對稱的)

For an **undirected graph** the **degree** of any vertex i is its row sum. (對於無向圖, 任一頂點的分支度為其各列之和)

For an **directed graph** the row sum is its **out-degree** and the column sum is its **in-degree**.

(對於有向圖, 各列之和為出分支度, 各行之和為入分支度)

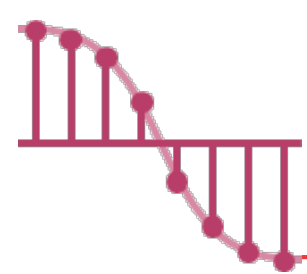
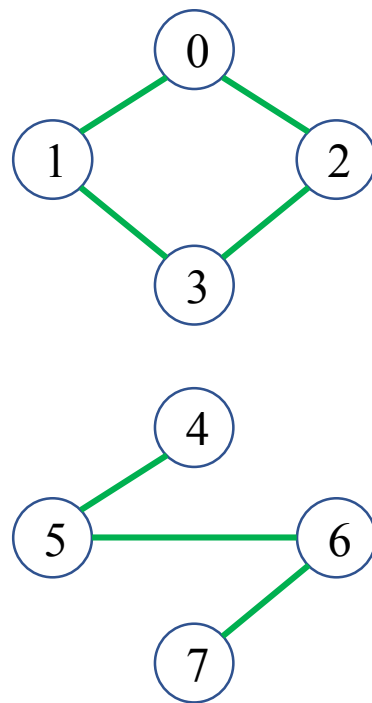


Figure 6.7: Adjacency matrices _{2/2}



G_4

	0	1	2	3	4	5	6	7
0	0	1	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0
2	1	0	0	1	0	0	0	0
3	0	1	1	0	0	0	0	0
4	0	0	0	0	0	1	0	0
5	0	0	0	0	1	0	1	0
6	0	0	0	0	0	1	0	1
7	0	0	0	0	0	0	1	0

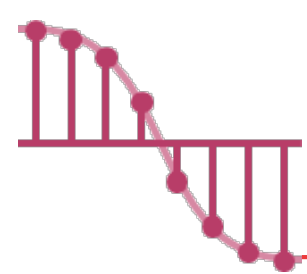
The adjacency matrix of G_4

The adjacency matrix for an undirected graph is symmetric. (無方向圖形的相鄰矩陣是對稱的)

For an undirected graph the degree of any vertex i is its row sum. (對於無向圖, 任一頂點的分支度為其各列之和)

For an directed graph the row sum is its out-degree and the column sum is its in-degree.

(對於有向圖, 各列之和為出分支度, 各行之和為入分支度)



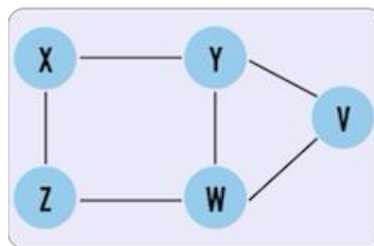
Graph Representations

➤ There are two most commonly used representations for graph:

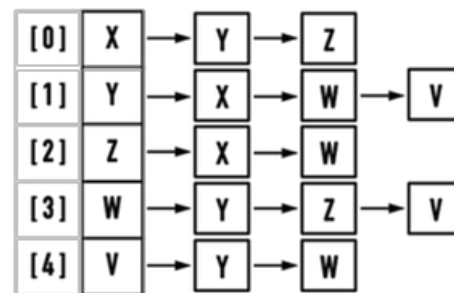
✓ Adjacency matrices

✓ Adjacency lists

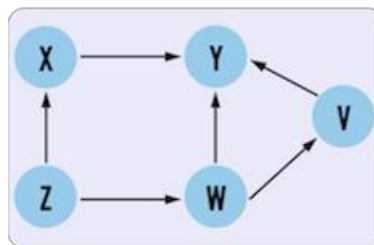
Undirected Graph



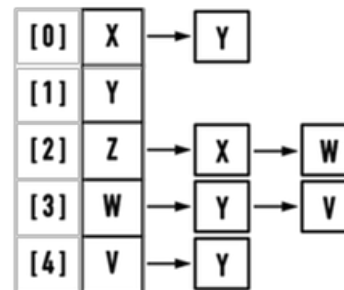
Adjacency List

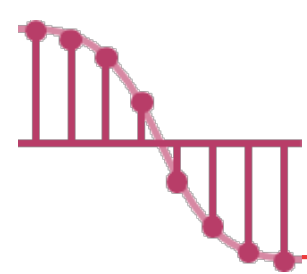


Directed Graph



Adjacency List





Adjacency Lists (相鄰串列)

- The n rows of the adjacency matrix are represented as n chains.
- There is one chain for each vertex in G .
- The node in chain i represent the vertices that are adjacent from vertex i .
- The **data** field of a chain node stores the index of an adjacent vertex.



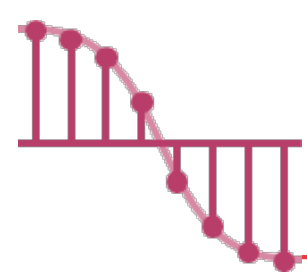
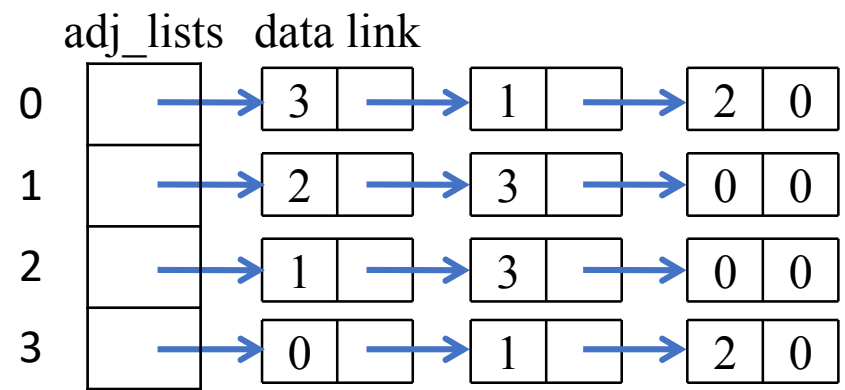
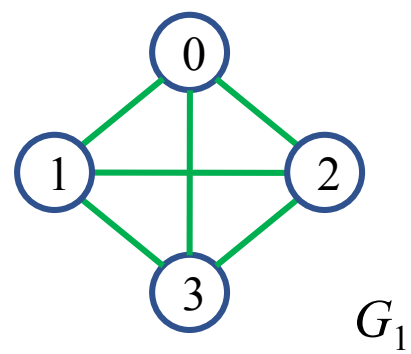
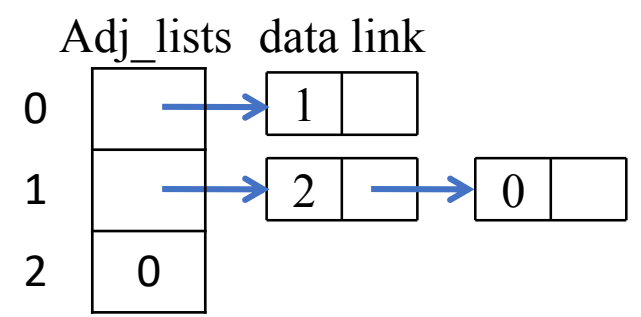
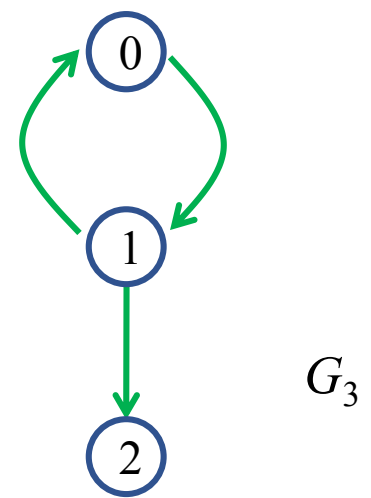


Figure 6.8: Adjacent lists



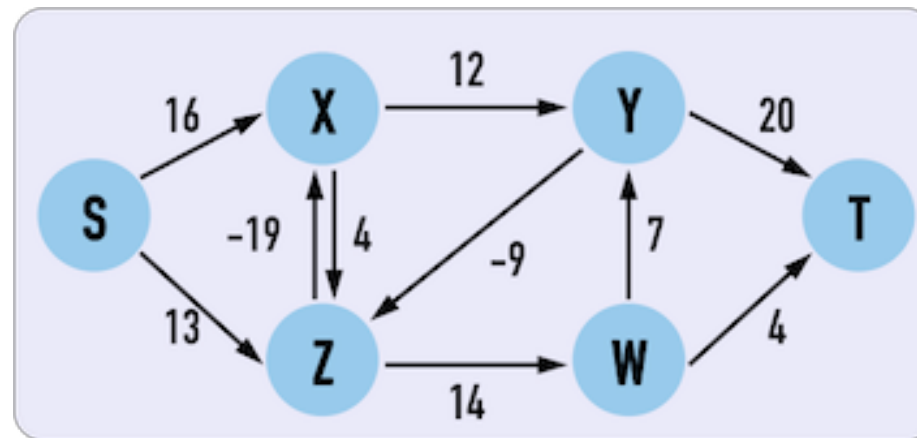
Adjacent list of G_1

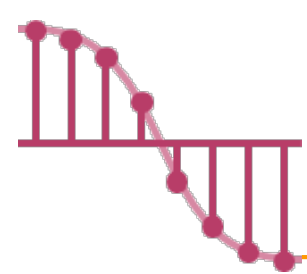


Adjacent list of G_3

Weighted Edges

- In many applications, the edges of a graph have **weights** associated with them.
- These weights may represent the **distance** from one vertex to another or the **cost** of going from one vertex to an adjacent vertex.
 - ✓ The adjacency matrix entries $a[i][j]$ would keep this information.
 - ✓ When adjacency lists are used, the weight information may be kept in the list nodes by **including an additional field, *weight***.

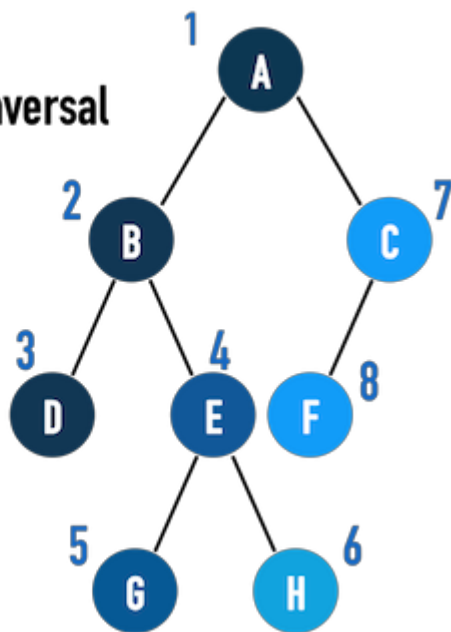




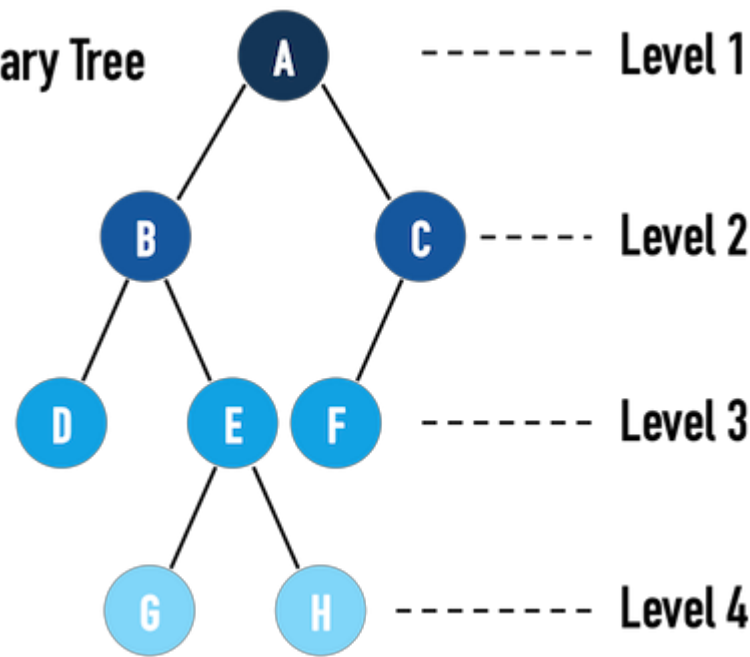
Elementary Graph Operations

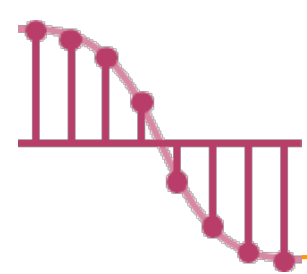
- Given an undirected graph, $G = (V, E)$, and a vertex v in $V(G)$, we wish to visit all vertices in G that are reachable from v .

Binary Tree:
Pre-Order Traversal



Binary Tree





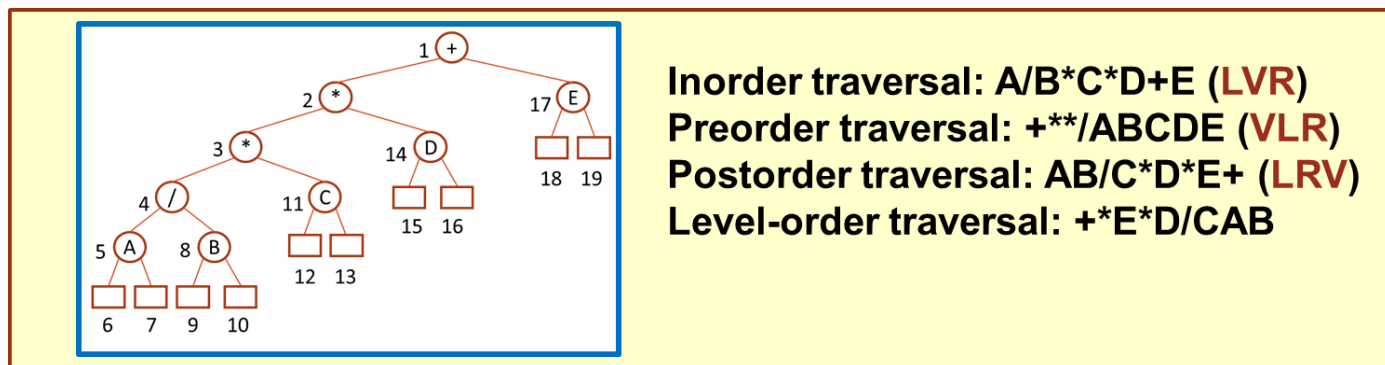
Elementary Graph Operations

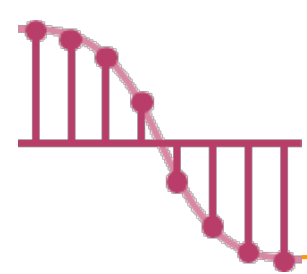
➤ We shall look at two ways of doing this:

✓ **Depth First Search** (DFS) is similar to a **preorder tree traversal**.

✓ **Breadth First Search** (BFS) is similar to a **level-order tree traversal**.

➤ In the following discussion, we shall assume that the linked adjacency list representation for graphs is used.

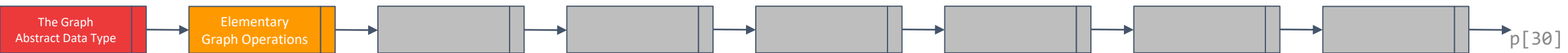
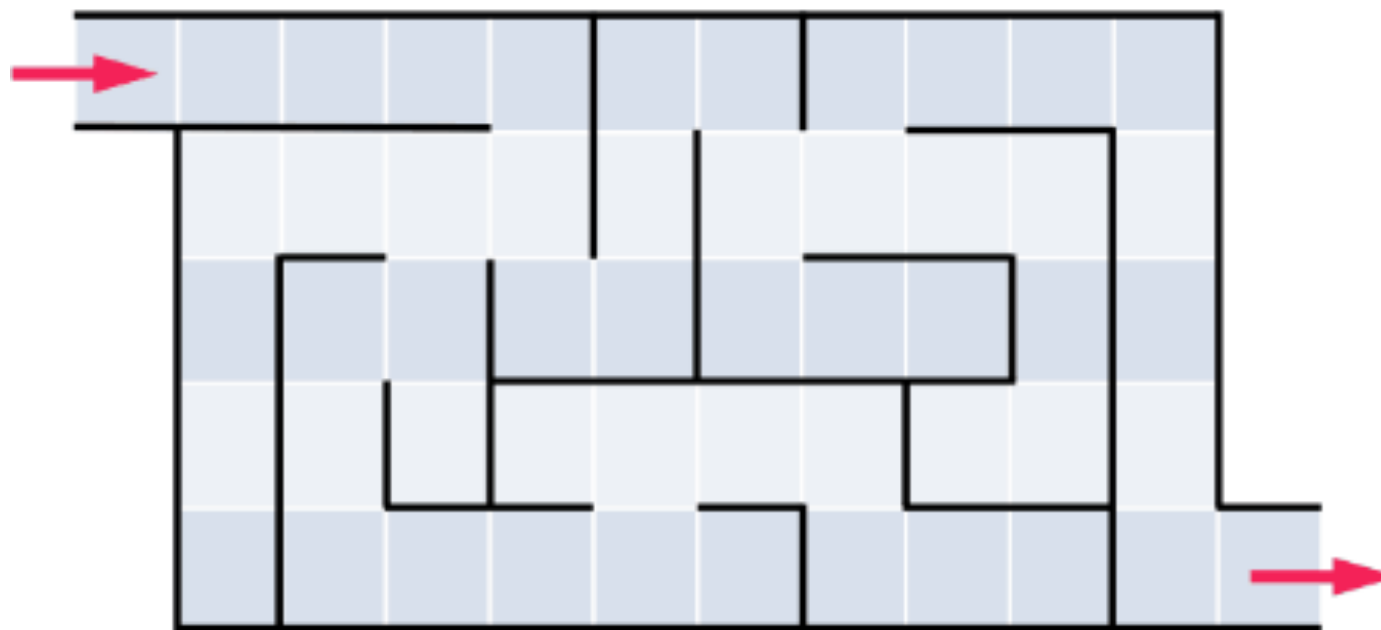


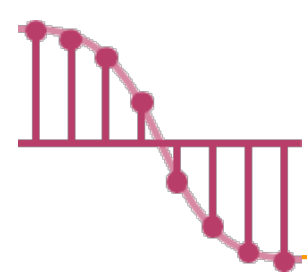


Before Depth First Search

➤ We consider a maze problem.

Maze Problem





Depth First Search

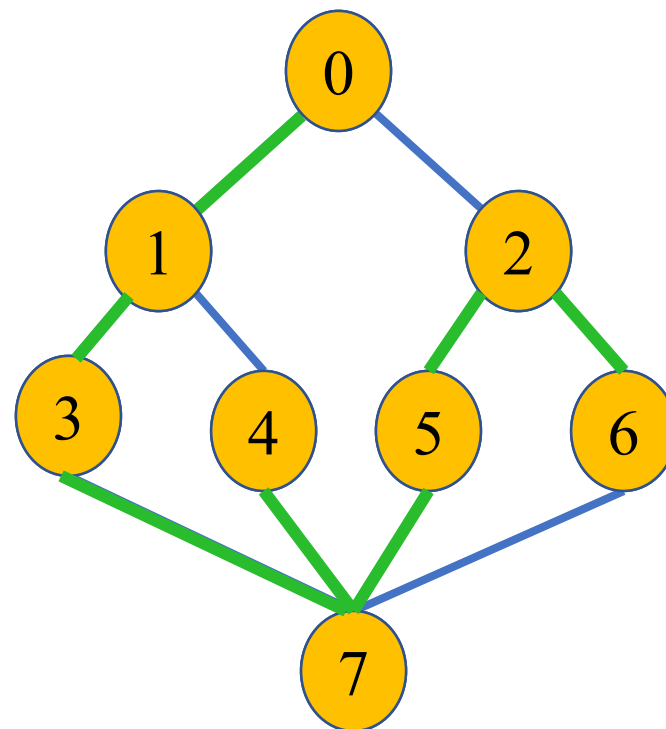
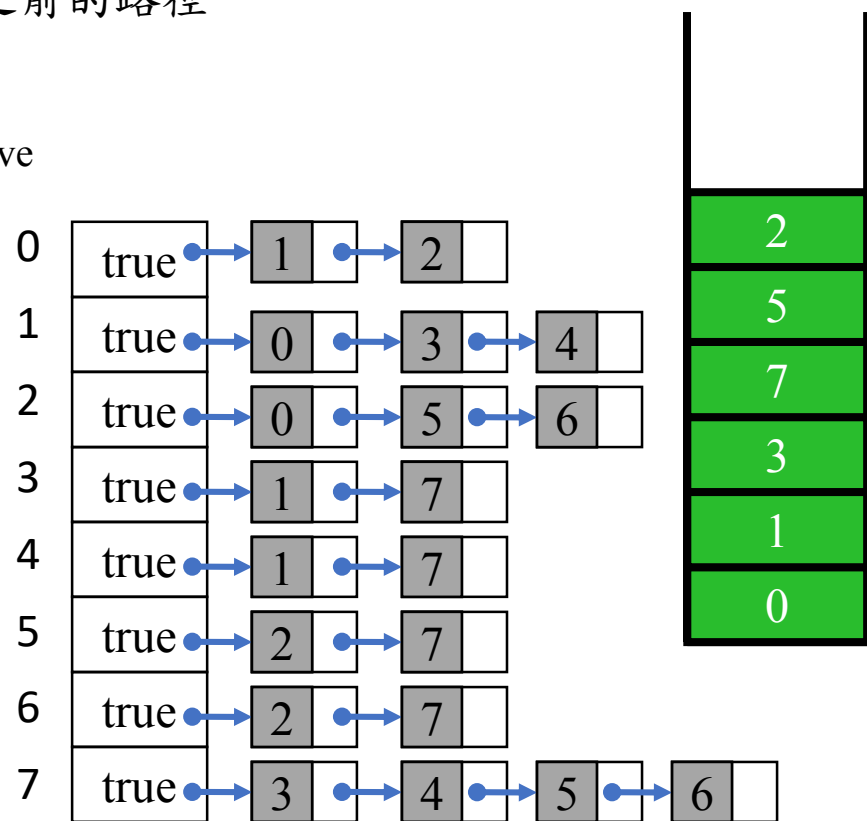
√LR

➤ 類似樹的**前序走訪**

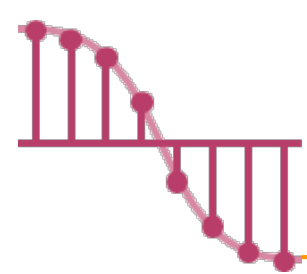
➤ 必須記錄之前的路徑

✓ Stack

✓ Recursive

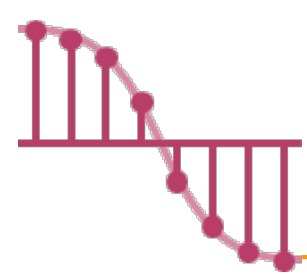


0 → 1 → 3 → 7 → 4 → 5 → 2 → 6



Depth First Search (DFS) _{1/2}

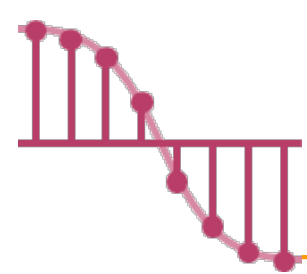
- We begin the search by visiting the start vertex, v .
- Next, we select an unvisited vertex, w , from v 's adjacency lists and carry out a depth first search on w .
- We preserve our current position in v 's adjacency list by placing it on a **stack**.
- Eventually our search reaches a vertex, u , that has no unvisited vertices on its adjacency list.



Depth First Search (DFS) _{2/2}

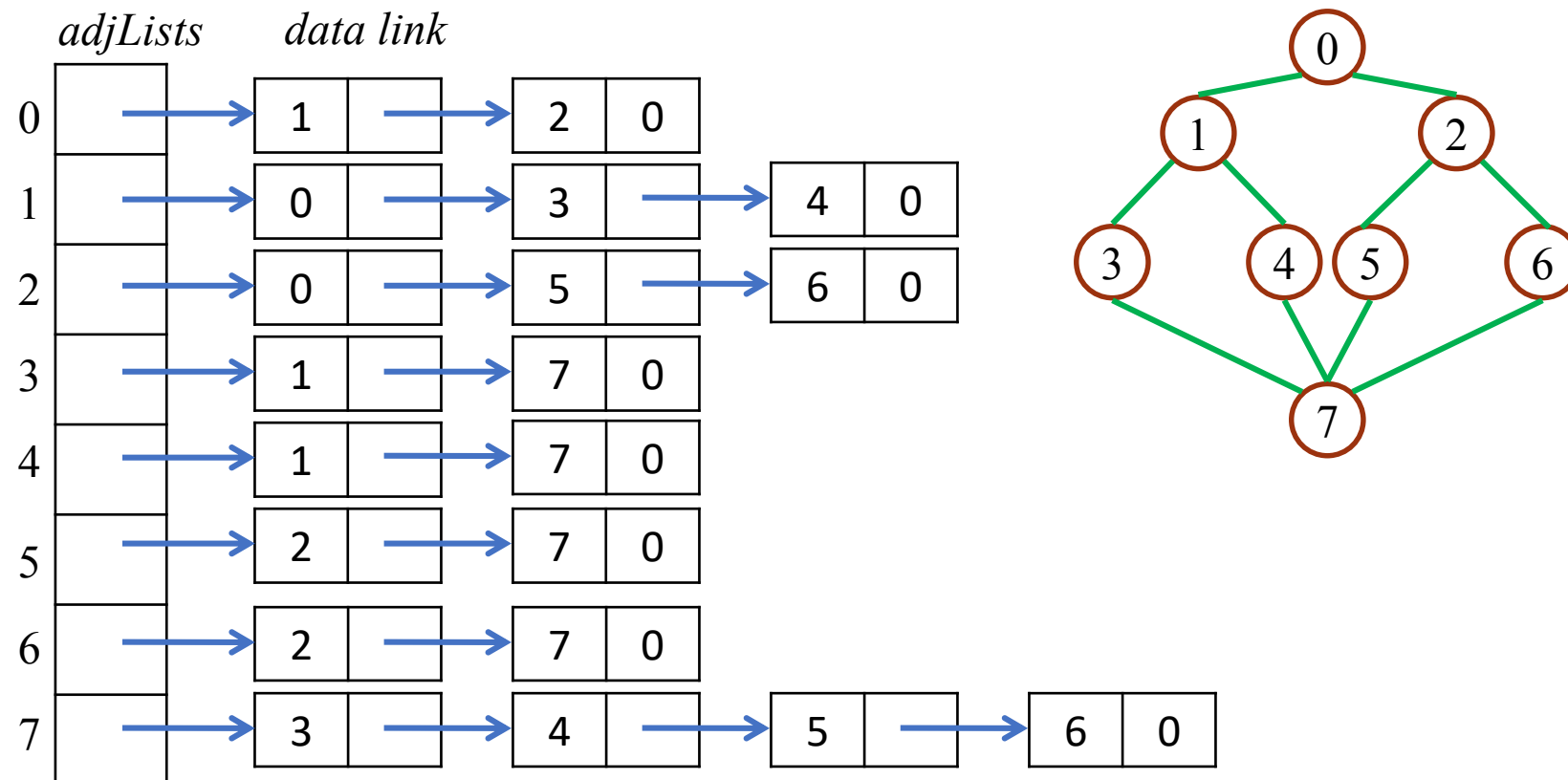
- At this point, we remove a vertex from **stack** and continue processing its adjacency list.
- Previously visited vertices are discarded; unvisited vertices are visited and placed on the stack.
- The search terminates when the stack is empty.

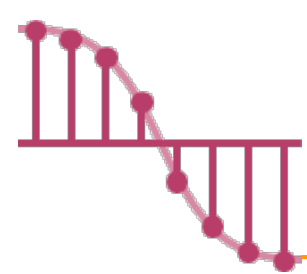




Depth First Search Order

➤ The depth first search order: $v_0, v_1, v_3, v_7, v_4, v_5, v_2, v_6$.



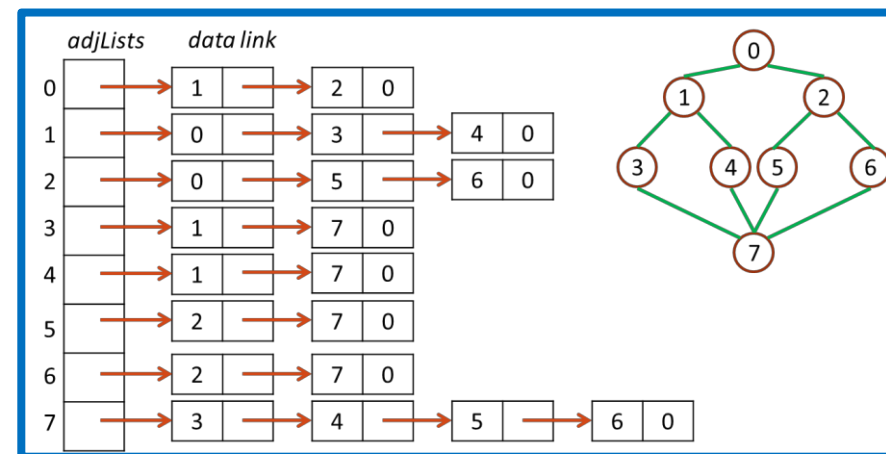


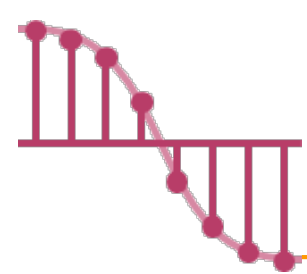
Program 6.1: Depth First Search

```
➤ #define FALSE 0
   #define TRUE 1
   short int visited[MAX_VERTICES]; /*int: FALSE */
```

```
void dfs(int v)
{
    /* depth first search of a graph beginning with vertex v. */
    nodePointer w;
    visited[v] = true;
    printf("%5d",v);
    for(w = graph[v]; w; w = w->link)
        if (!visited[w->vertex])
            dfs(w->vertex);
}
```

w不為NULL

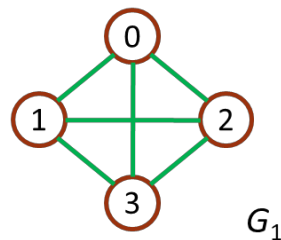




Analysis of DFS

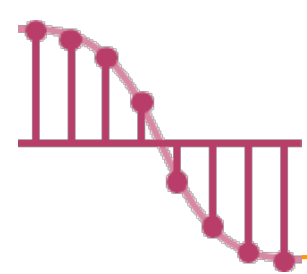
- If we represent G by its adjacency lists, then we can determine the vertices adjacent to v by following a chain of links.
- Since DFS examines each node in the adjacency lists at most once, the time to complete the search is $O(e)$.
- If we represent G by its adjacency matrix, then determining all vertices adjacent to v requires $O(n)$ time.
 - ✓ Because we visit at most n vertices, the total time is $O(n^2)$.

Example.



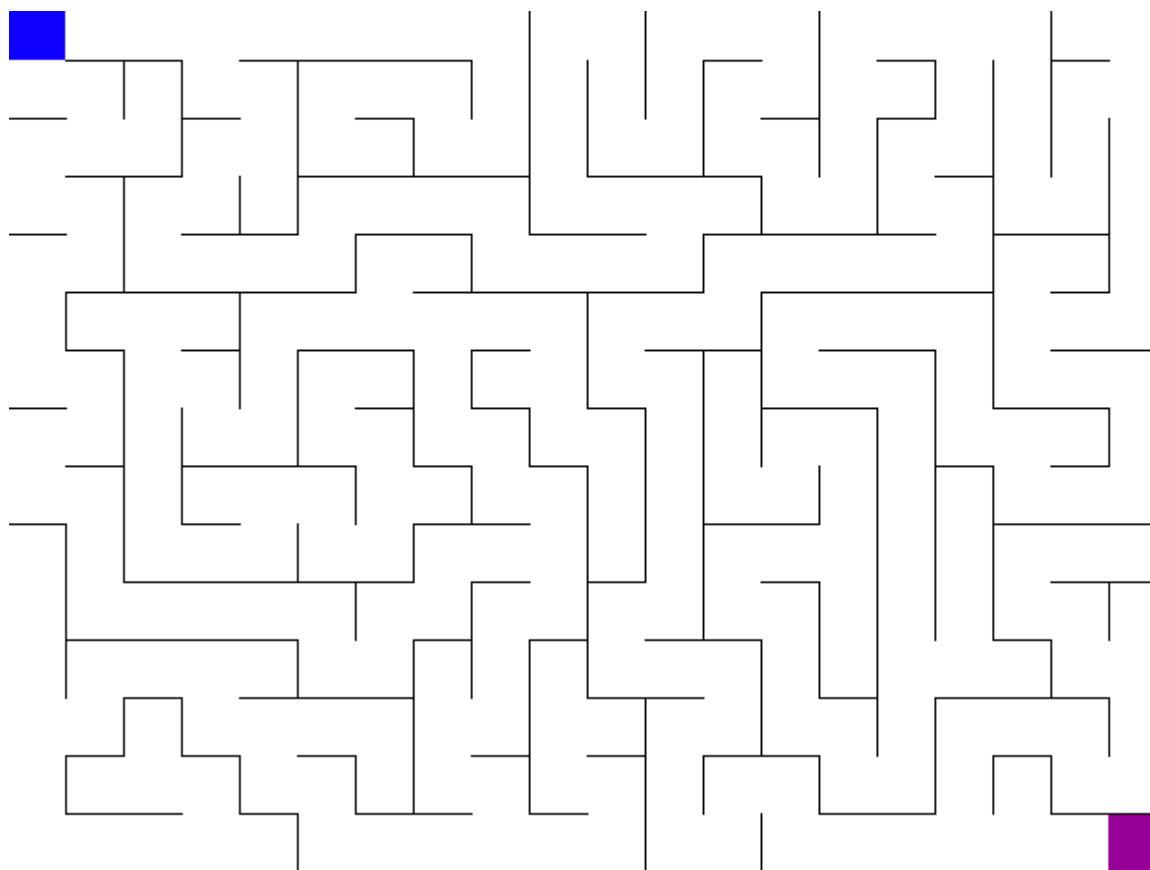
0	0	1	1	1
1	1	0	1	1
2	1	1	0	1
3	1	1	1	0

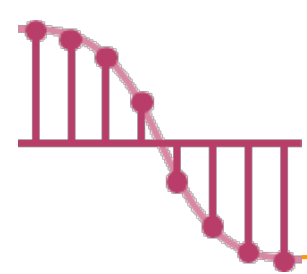
The adjacency matrix of G_1



Before Breadth First Search

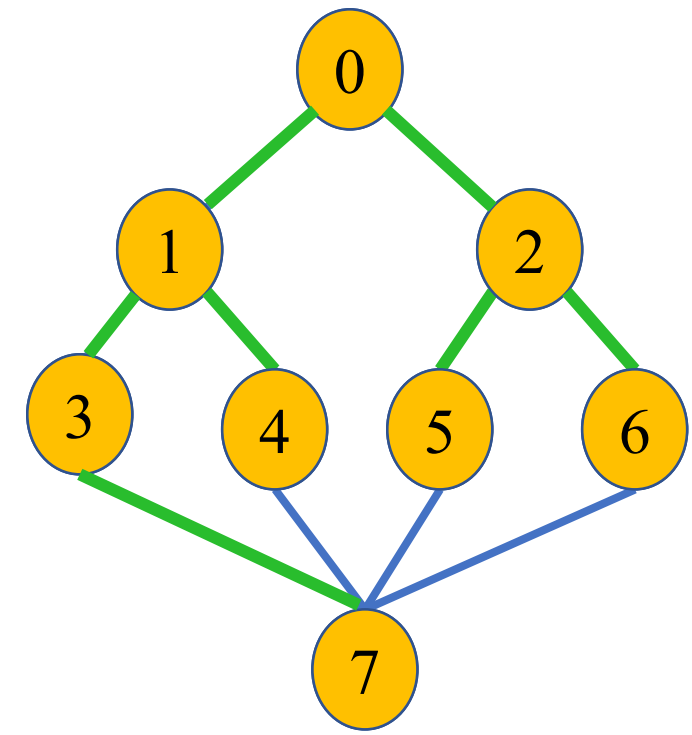
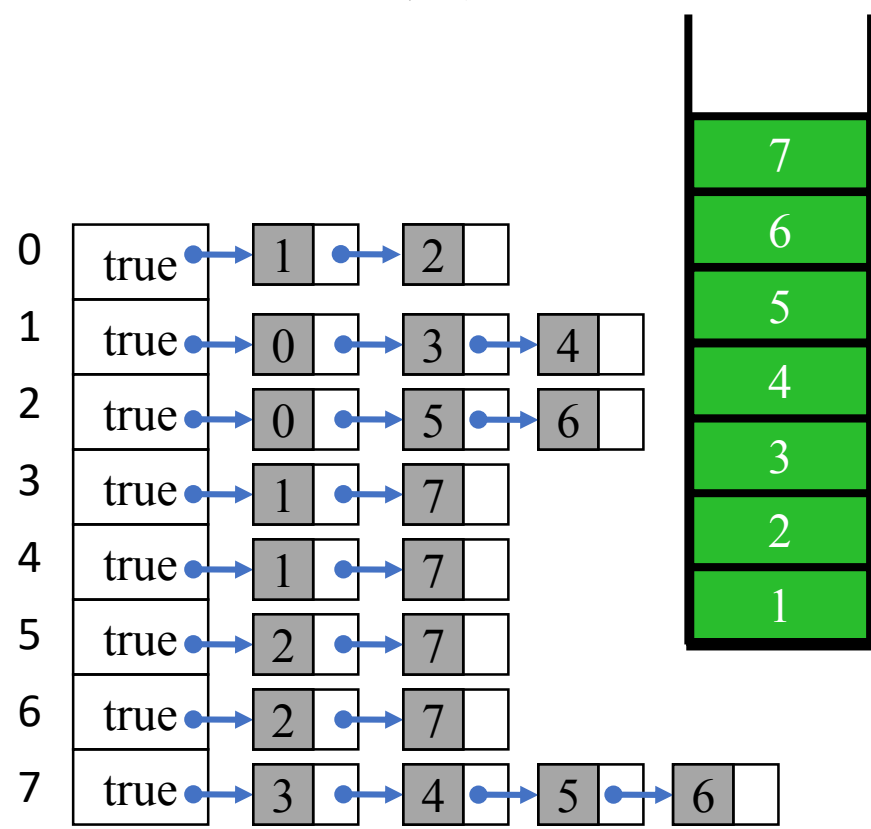
➤ We consider a maze problem.



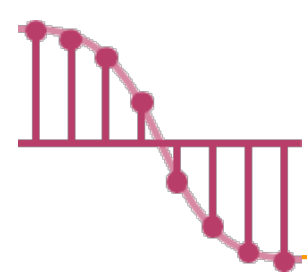


Breadth First Search

- 類似樹的**階序走訪**
- 未經走訪的頂點被走訪後放入佇列中
 - ✓ Queue
 - ✓ Iterative

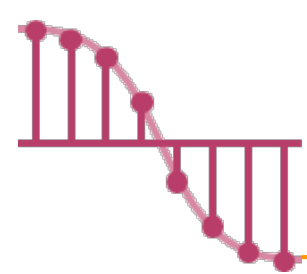


0 → 1 → 2 → 3 → 4 → 5 → 6 → 7



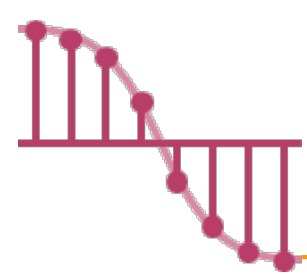
Breadth First Search (BFS) _{1/2}

- Breadth First Search starts at vertex v and marks it as visited.
- Then visiting each of the vertices on v 's adjacency list.
- When we have visited all the vertices on v 's adjacency list, we visit all the unvisited vertices that are adjacent to the first vertex on v 's adjacency list.
- To implement this scheme, as we visit each vertex we place the vertex in a **queue**.



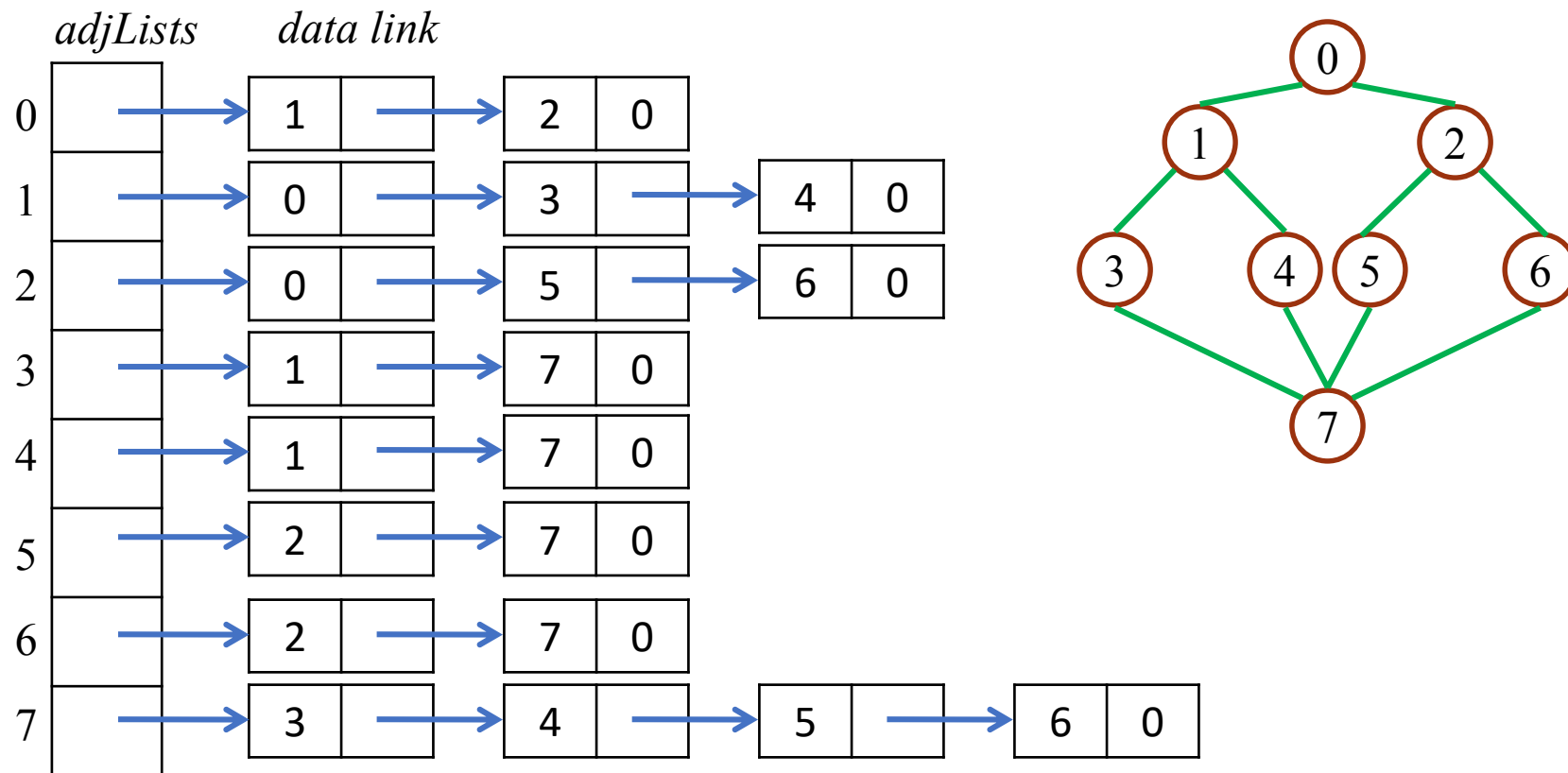
Breadth First Search (BFS) _{2/2}

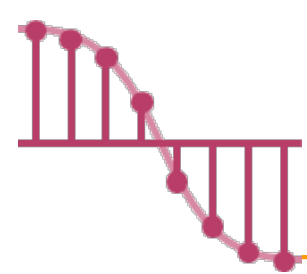
- When we have exhausted an adjacency list, we remove a vertex from the queue and proceed by examining each of the vertices on its adjacency list.
- **Unvisited vertices are visited and placed on the queue; visited are ignored.**
- We have finished the search when the queue is empty.



Breadth First Search Order

➤ The breadth first search order: v0, v1, v2, v3, v4, v5, v6, v7.



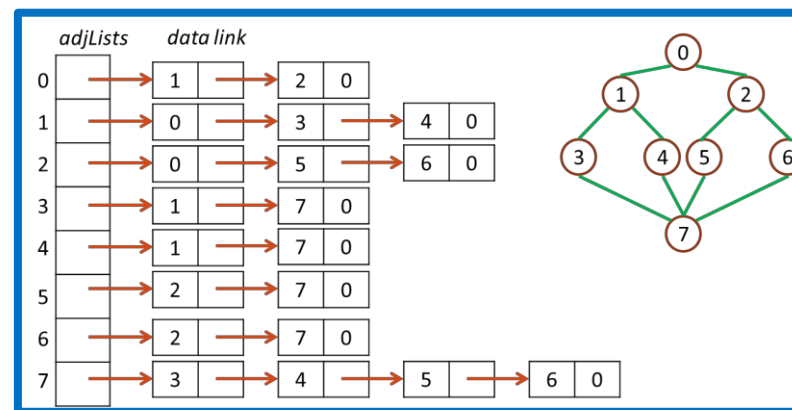


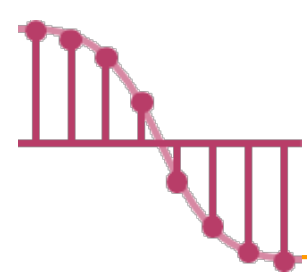
Program 6.2: Breadth First Search of a Graph

```

➤ void bfs(int v)
{
    /* breadth first traversal of a graph, starting with node v
    the global array visited is initialized to 0, the queue
    operations are similar to those described in Chapter 4. */
    nodePointer w;
    front = rear = NULL; /* initialize queue */
    printf("%5d", v);
    visited[v] = TRUE;
    addq(v);
    while(front) {
        v = deleteq();
        for(w = graph[v]; w ; w->link)
            if(!visited[w->vertex]) {
                printf("%5d", w->vertex);
                addq(w->vertex);
                visited[w->vertex]=TRUE;
            }
    }
}

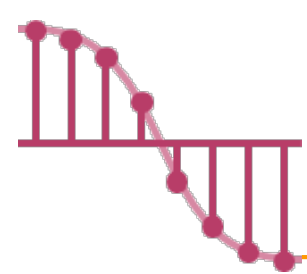
```





Analysis of BFS

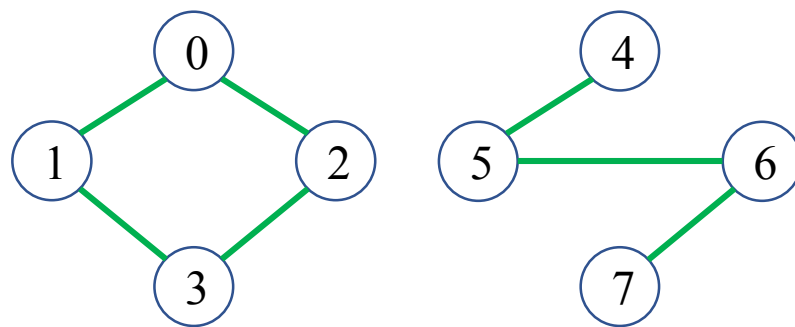
- Since each vertex is placed on the queue exactly once, the **while** loop is iterated at most n times.
- For the **adjacency list** representation, this loop has a total cost of $d_0 + \dots + d_{n-1} = O(e)$, where $d_i = \text{degree}(v_i)$.
- For the **adjacency matrix** representation, the while loop takes $O(n)$ time for each vertex visited.
 - ✓ Therefore, the total time is $O(n^2)$.
- As was true of DFS, all vertices visited, together with all edges incident to them, form a connected subgraph of G .



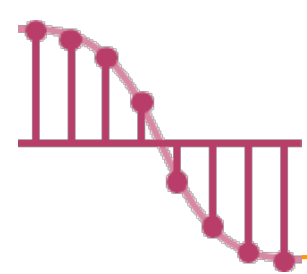
Connected ?

➤ Let us look at the problem of determining whether or not an undirected graph is connected.

✓ We can implement this operation by calling either *dfs* (0) or *bfs* (0) and then determining if there are any unvisited vertices.

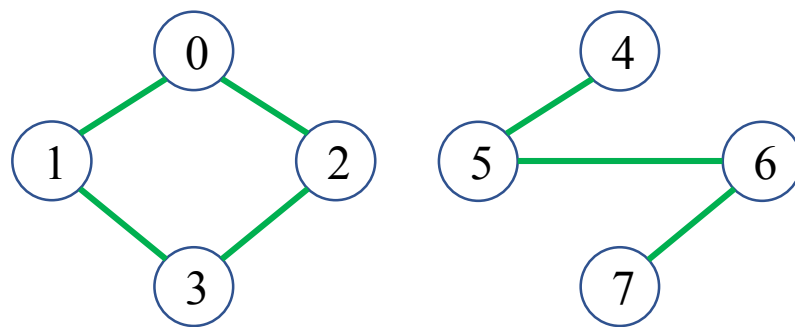


G_4

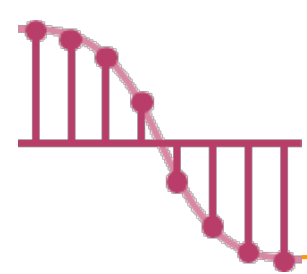


Connected Components

- A related problem is that of listing the connected components of a graph.
 - ✓ This can be done by making repeated calls to either $dfs(v)$ or $bfs(v)$ where v is an unvisited vertex.

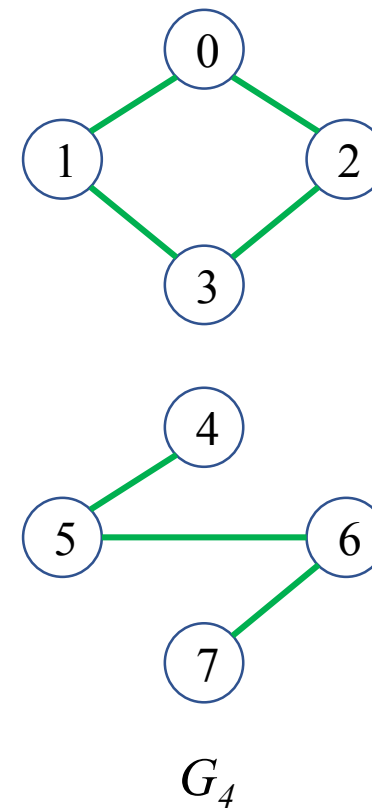


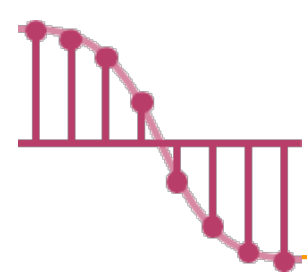
G_4



Program 6.3: Connected Components

```
➤ void connected(void) /*dfs(0) or bfs(0) */  
{  
    /*determine the connected components of a graph */  
    int i;  
    for (i=0; i<n; i++)  
    {  
        if(!visited[i]) {  
            dfs(i);  
            printf("\n");  
        }  
    }  
}
```



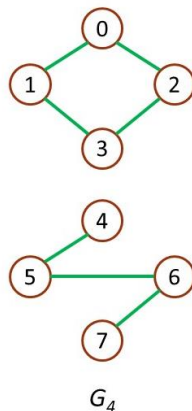


Analysis of Connected

- If G is represented by its adjacency lists, then the total time taken by DFS is $O(e)$.
- Since the **for** loop takes $O(n)$ time, the total time needed to generate all the connected components is $O(n+e)$.
- If G is represented by its adjacency matrix, then the time needed to determine the connected components is $O(n^2)$.

Program 6.3: Connected Components

```
void connected(void) /*dfs(0) or bfs(0) */
{
    /*determine the connected components of a graph */
    int i;
    for (i=0; i<n; i++)
    {
        if(!visited[i]) {
            dfs(i);
            printf("\n");
        }
    }
}
```

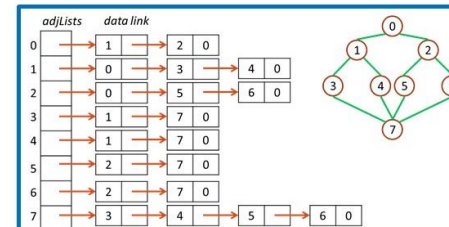


Program 6.1: Depth First Search

```
#define FALSE 0
#define TRUE 1
short int visited[MAX_VERTICES]; /*int: FALSE */

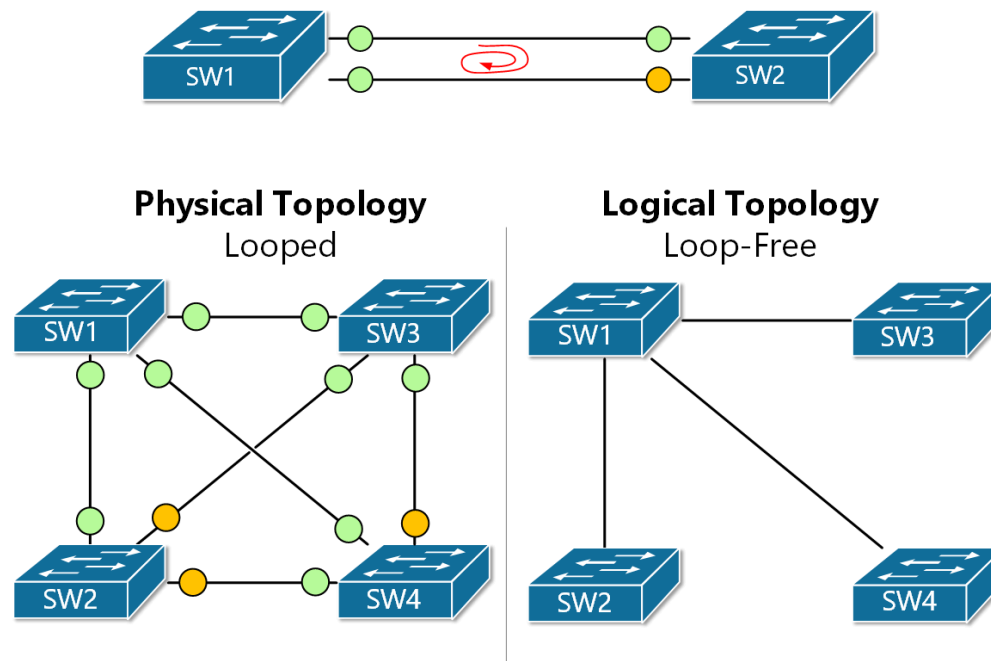
void dfs(int v)
{
    /* depth first search of a graph beginning with vertex v. */
    nodePointer w;
    visited[v] = true;
    printf("%5d",v);
    for(w = graph[v]; w; w = w->link)
        if (!visited[w->vertex])
            dfs(w->vertex);
}
```

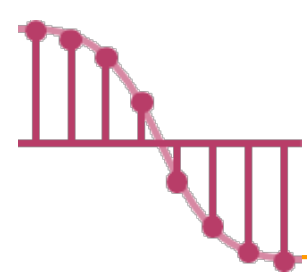
w不為NULL



Before Spanning Trees

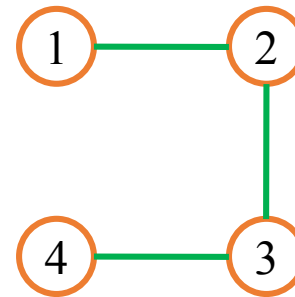
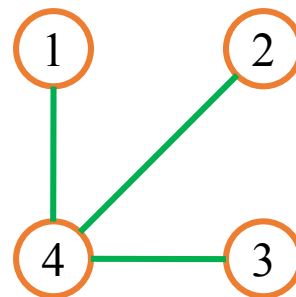
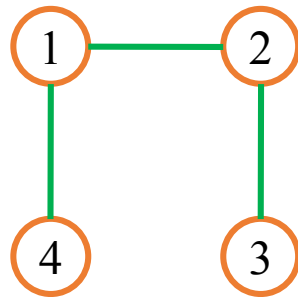
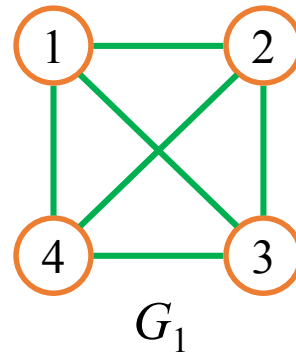
- If we have clients connected to SW1 and clients connected to SW2 we would like to have a **redundant connection** between the switches.



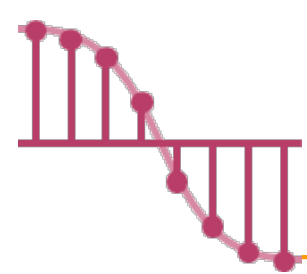


Spanning Trees

➤ A tree T is said to be a **spanning tree** of a connected graph G if T is a subgraph of G and T contains all vertices of G .



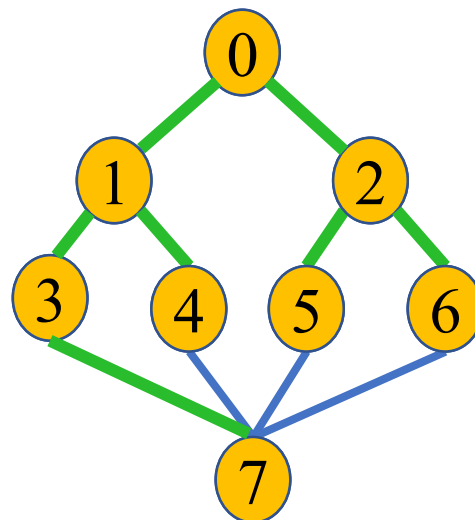
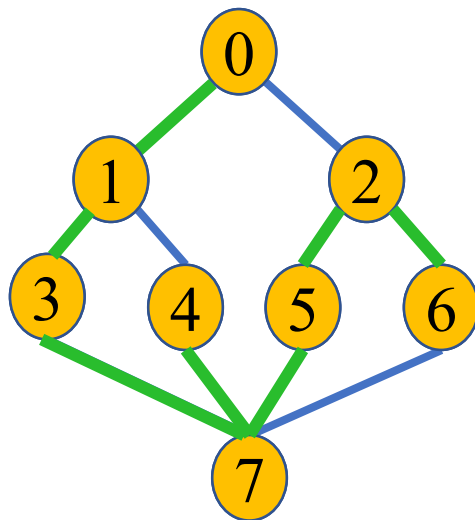
Three spanning trees of G_1

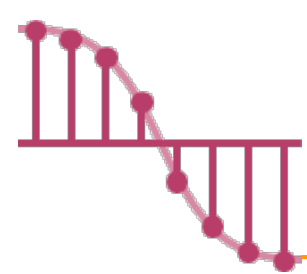


Spanning Trees with DFS or BFS

➤ When graph G is connected, a **DFS** or **BFS** implicitly partitions the edges in G into two sets:

- ✓ Tree edges: the set of edges used or traversed during the search.
- ✓ Nontree edges : the set of remaining edges.

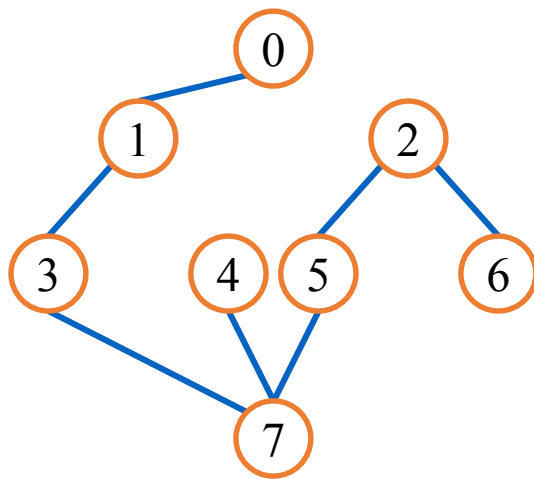
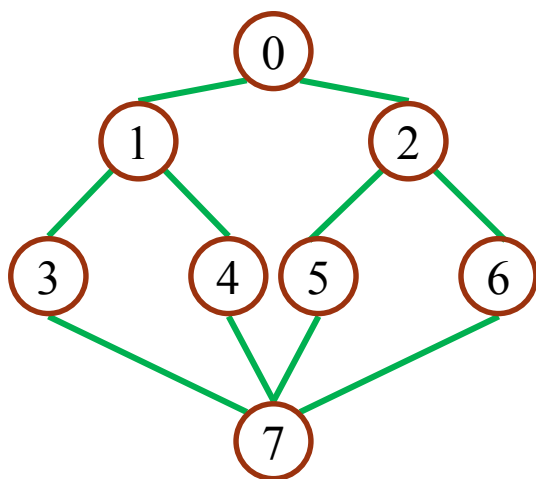




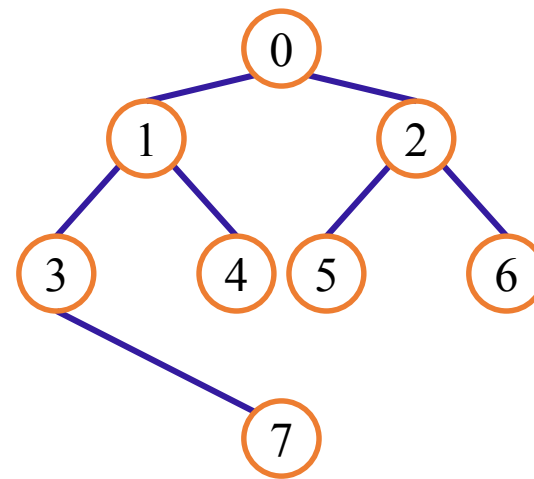
Depth-First and Breadth-First Spanning Trees

- When DFS is used, the resulting spanning tree is known as a **depth first spanning tree**.
- When BFS is used, the resulting spanning tree is known as a **breadth first spanning tree**.

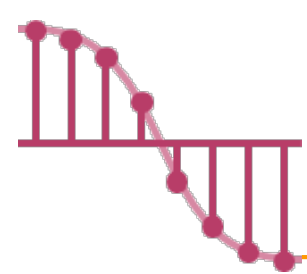
有 n 个点, 轉為生成樹時, 必有 $n-1$ 條邊



DFS (0)
spanning tree

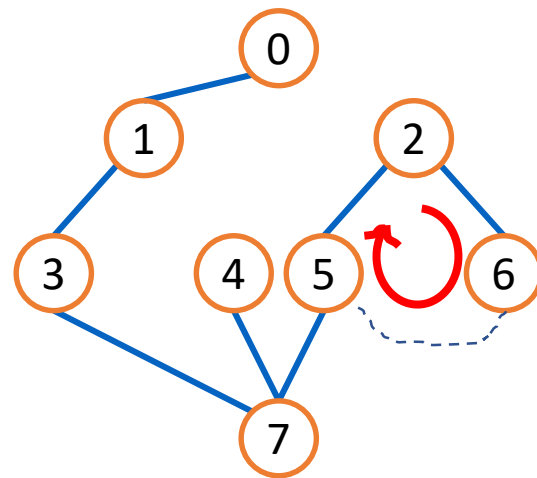


BFS (0)
spanning tree

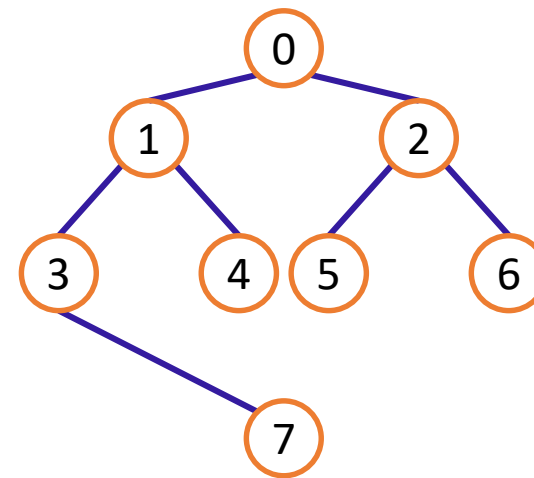


Properties of spanning trees

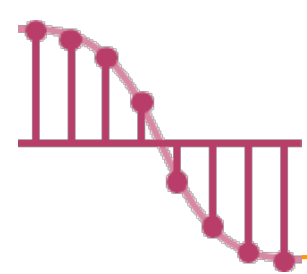
➤ **Property 1:** Suppose we add a nontree edge, (v, w) , into any spanning tree, T . The result is a cycle that consists of the edge (v, w) and all the edges on the path from w to v in T .



DFS (0)
spanning tree



BFS (0)
spanning tree

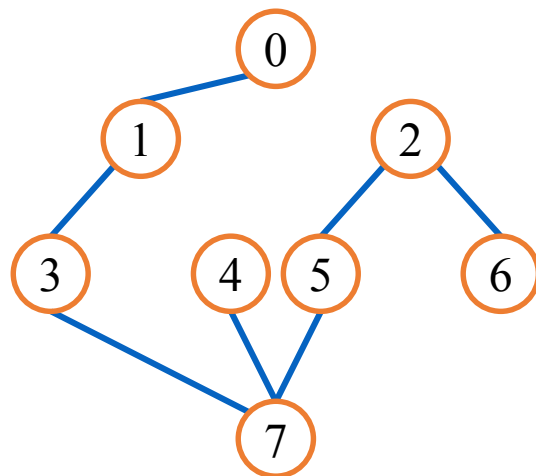


Properties of spanning trees

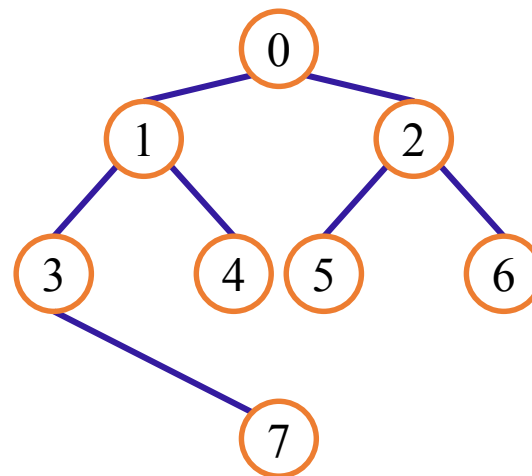
➤ **Property 2:** A spanning tree is a minimal subgraph, G' , of G such that $V(G') = V(G)$ and G' is connected.

✓ We define a minimal subgraph as one with the fewest number of edge.

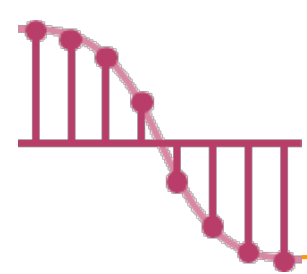
✓ A spanning tree has $n-1$ edges.



DFS (0)
spanning tree



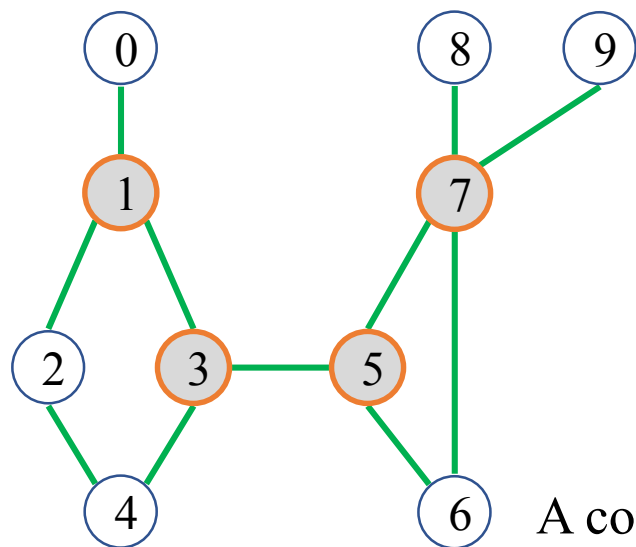
BFS (0)
spanning tree



Articulation Points and Biconnected Graph

➤ An **articulation point** is a vertex v of G such that the deletion of v , together with all edges incident on v , produces a graph, G' , that **has at least 2 connected components**.

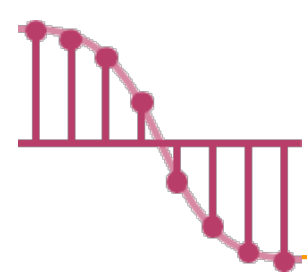
✓ Fig 6.19 has four articulation points, vertices 1, 3, 5, and 7.



A connected graph

關節点会是图中一节点 v 。
若將 v 即其所有与之相連的边删除產生的新图 G'
 G' 中至少包含两个連通分量

**How to find
articulation points?**

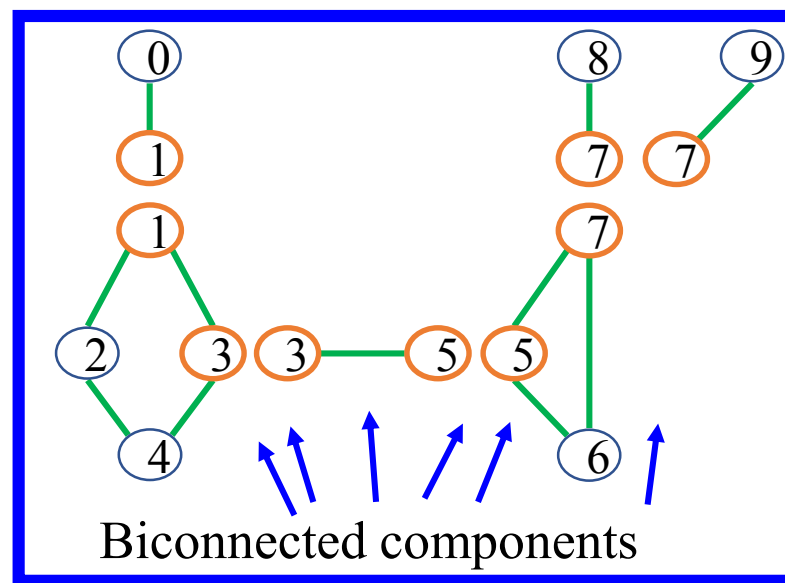
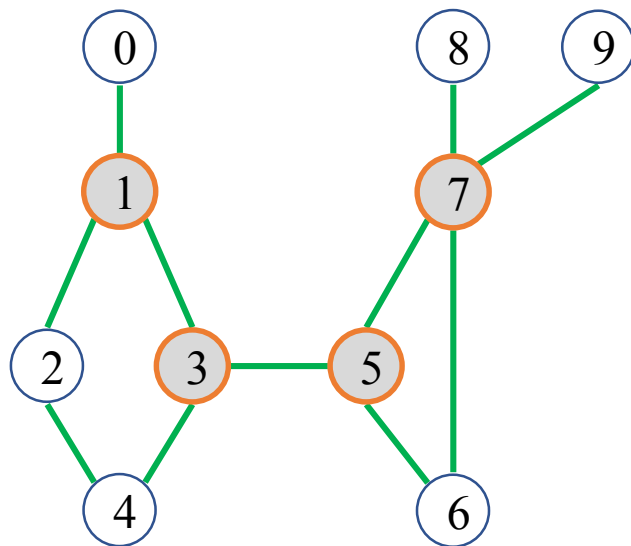


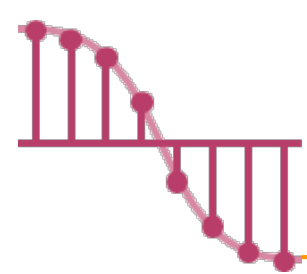
Articulation Points and Biconnected Graph

關節点

双连通分量

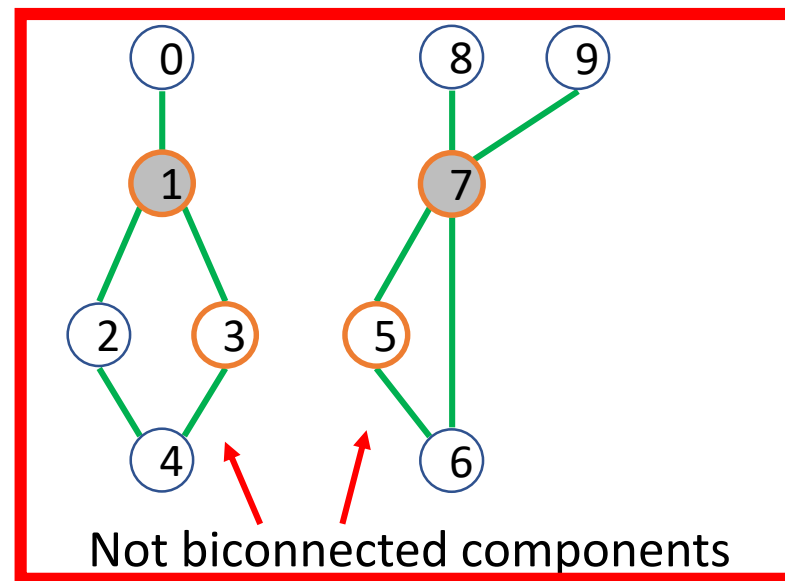
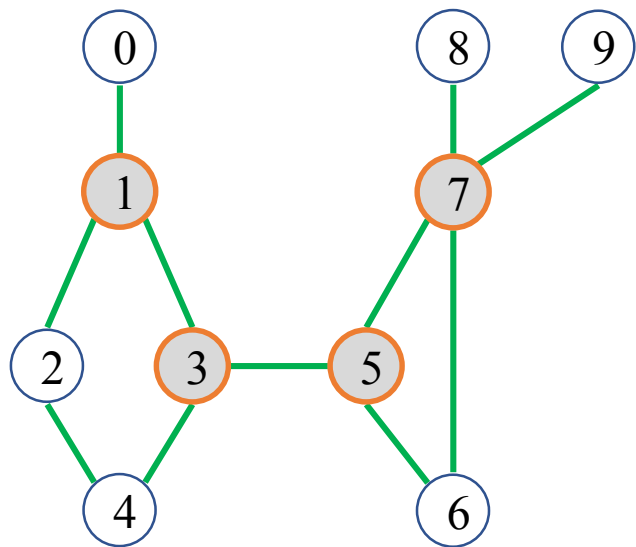
- A **biconnected graph** is a connected graph that has no articulation points.
- A **biconnected component** of a connected graph G is a **maximal biconnected subgraph** H of G .
 - ✓ By maximal, we mean that G contains no other subgraph that is both biconnected and properly contains H .

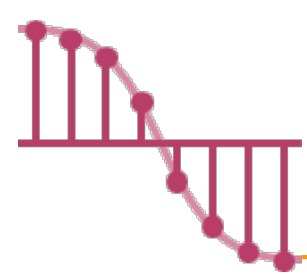




Articulation Points and Biconnected Graph

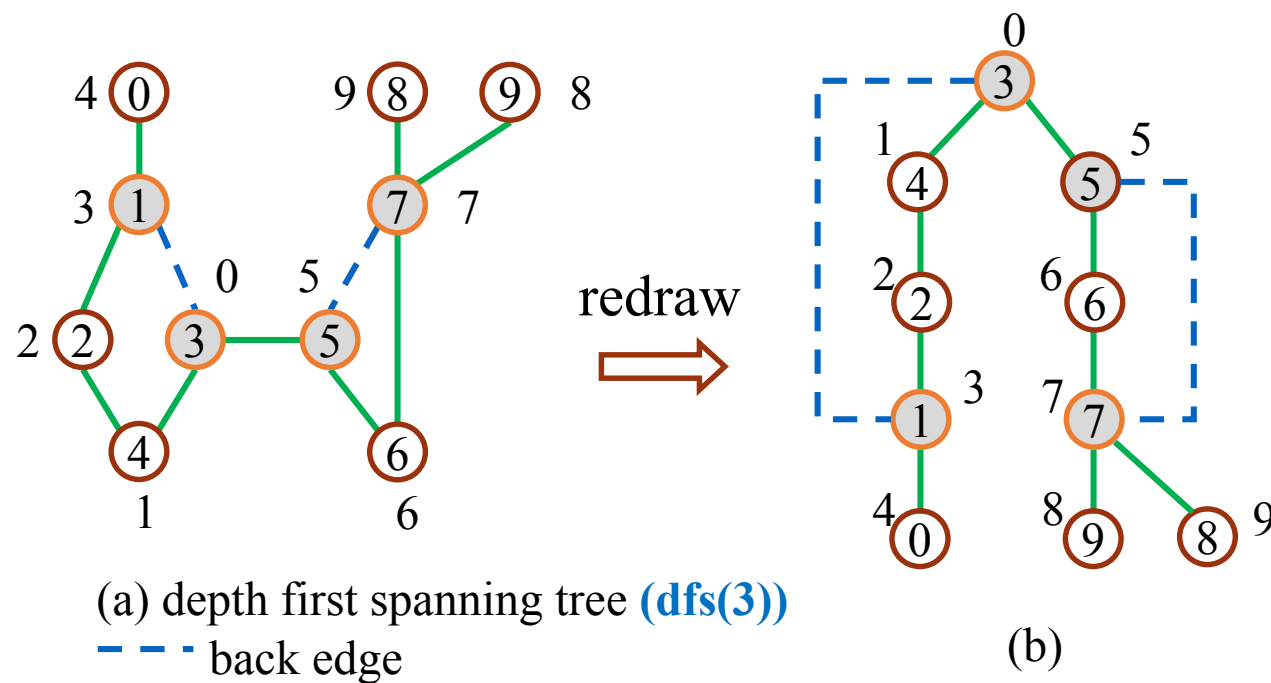
- A **biconnected graph** is a connected graph that has no articulation points.
- A **biconnected component** of a connected graph G is a **maximal biconnected subgraph** H of G .
 - ✓ By maximal, we mean that G contains no other subgraph that is both biconnected and properly contains H .

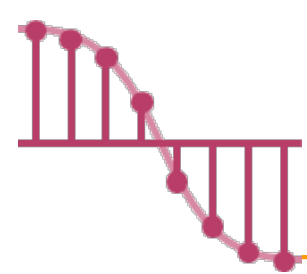




Find the articulation points _{1/3}

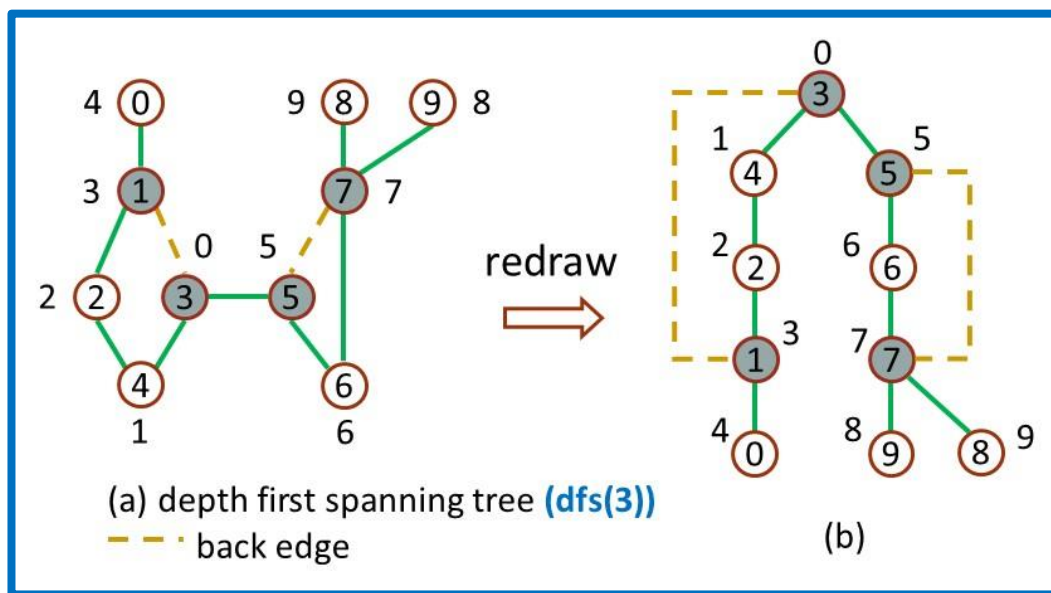
- We can find the biconnected components of G by using any depth first spanning tree of G .
- The broken lines in Fig 6.20(b) represent **nontree edges**. We now call these **nontree edges the back edges**.





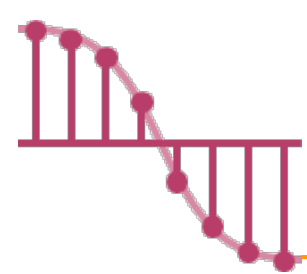
(Two Observations)

- The root of a depth first spanning tree is an articulation point **iff** it has at least two children.
- Any other vertex u is an articulation point **iff** it has at least one **child** w such that we cannot reach an ancestor of u using that consists of only w , descendants of w , and a single back edge.



頂點 5 是一個連接點：存在子頂點 6，我們無法僅用頂點 6，頂點 6 的後代和單一個後退邊到達頂點 5 的祖先。

頂點 6 不是一個連接點：所有頂點 6 的子頂點(只有頂點 7)，我們可以用頂點 7，頂點 7 的後代和單一個後退邊到達頂點 5(頂點 6 的某個祖先)。



Find the articulation points ^{2/3}

➤ The **depth first numbers**, or **dfn**, of the vertices give the sequence in which the vertices are visited during the depth first search.

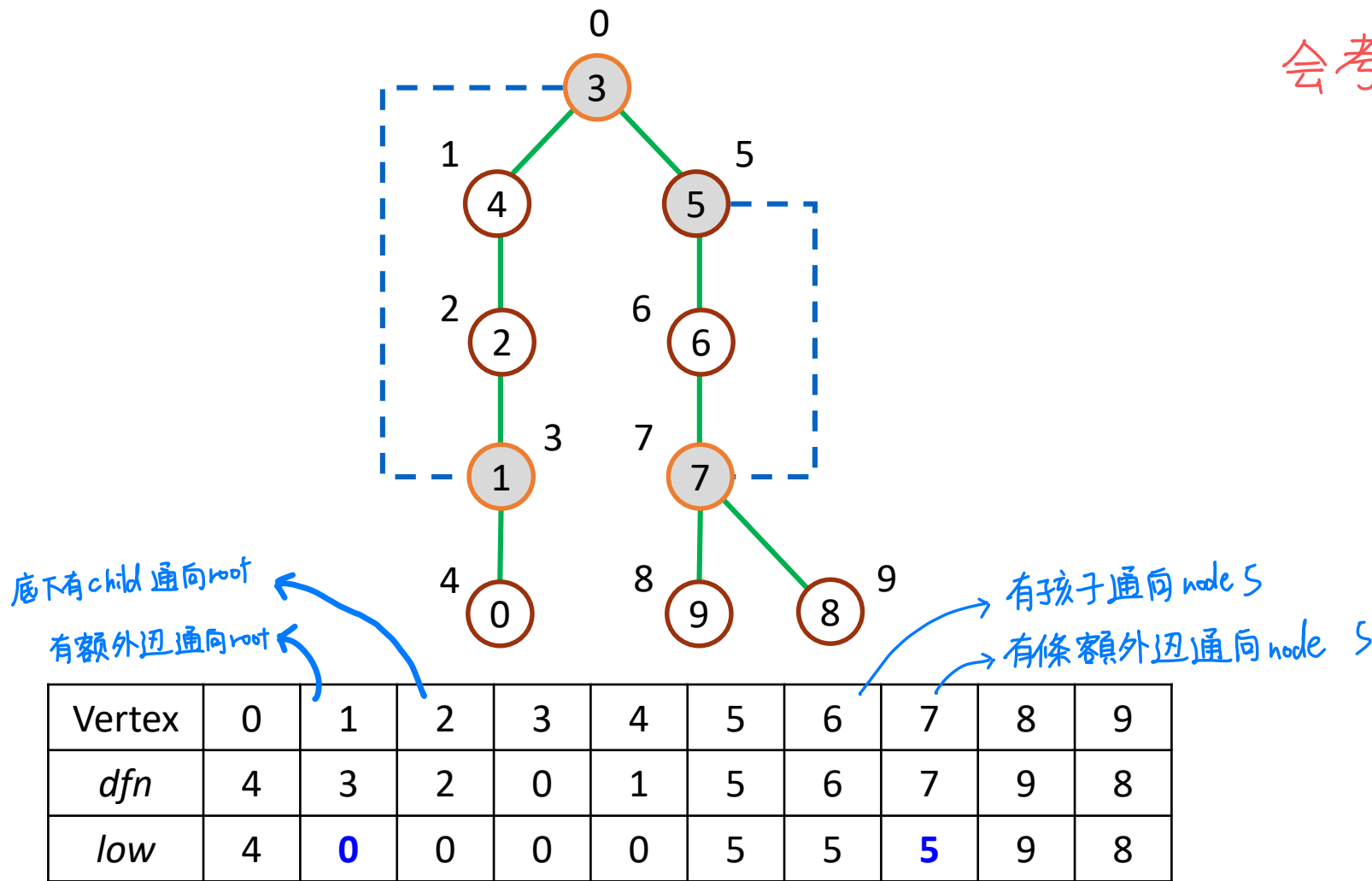
✓ If u is an ancestor of v in the depth first spanning tree, then
 $dfn(u) < dfn(v)$.

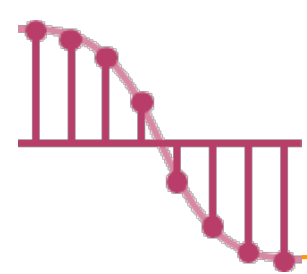
➤ The **low** (u) value of vertex u is the lowest depth first number that we can reach from u using a path of descendants followed by at most 1 back edge:

$$low(u) = \min \{ \begin{array}{l} dfn(u), \\ \min \{ low(w) \mid w \text{ is a child of } u \}, \\ \min \{ dfn(w) \mid (u, w) \text{ is a back edge} \} \end{array} \}$$

Figure 6.21:

dfn and *low* values for *dfs* spanning tree with root=3

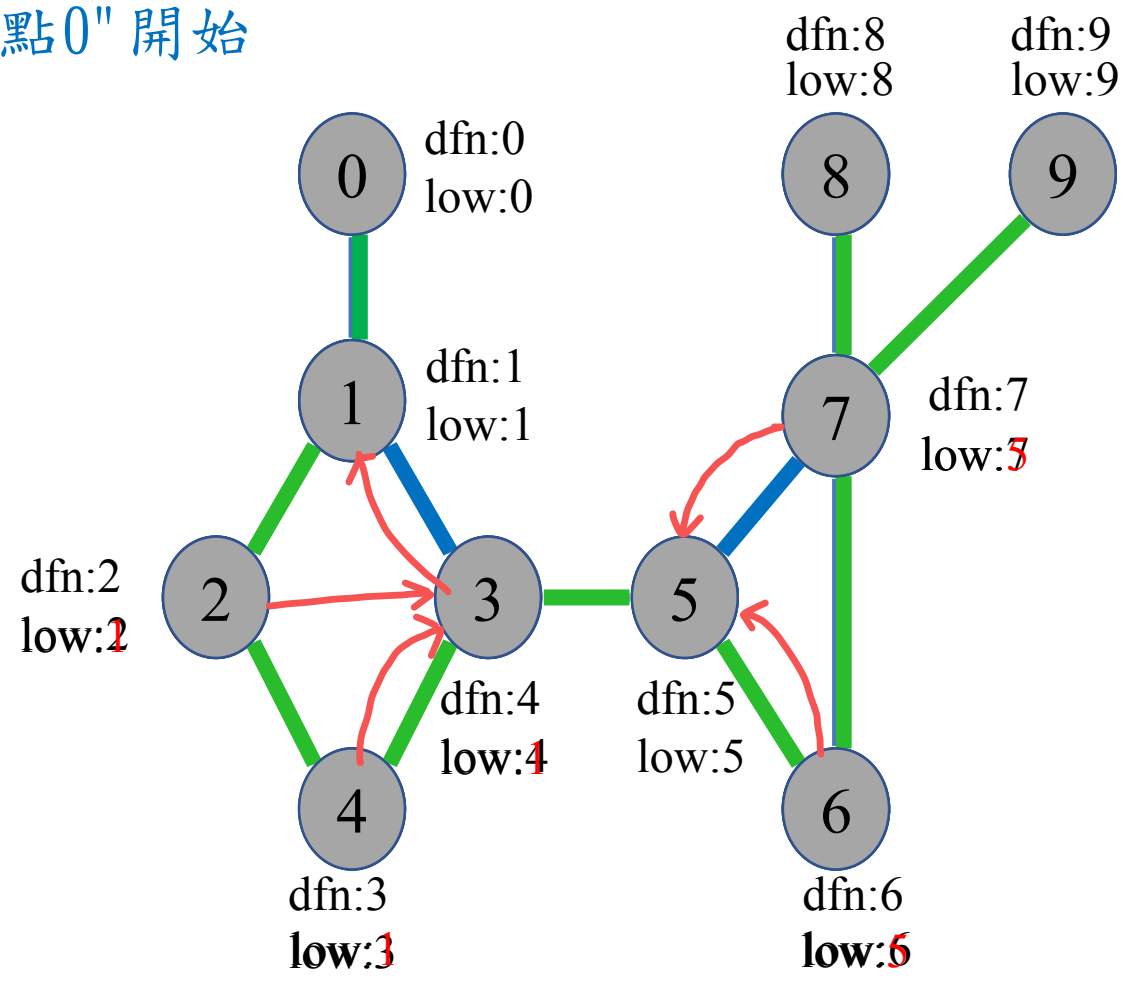




Determining *dfn* and *low*

➤ 假設這次由"頂點0"開始

会考!



Program 6.4: Determining *dfn* and *low*

➤ #define MIN2 (x, y) ((x) < (y) ? (x) : (y))

short int dfn[MAX_VERTICES];

short int low[MAX_VERTICES];

int num = 0;

void **dfnlow**(int u, int v)

{
 /* compute dfn and low while performing a dfs search
 beginning at vertex u, v is the parent of u (if any) */

node_pointer ptr;

int w;

dfn[u] = low[u] = num++;

for (ptr = graph[u]; ptr; ptr = ptr->link) {

 w = ptr->vertex;

 if (dfn[w] < 0) { /*w is an unvisited vertex */

dfnlow(w, u);

 low[u] = MIN2(low[u], low[w]);

 }

 else if (w != v)

 low[u] = MIN2(low[u], dfn[w]);

}

}

➤ Time Complexity: $O(e)$

void init(void)

{

 int i;

 for(i = 0; i < n; i++) {

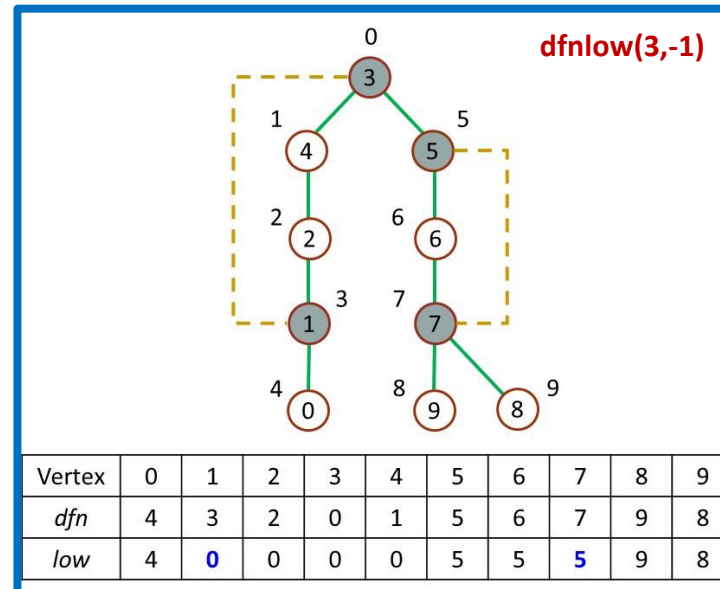
 visited[i] = FALSE;

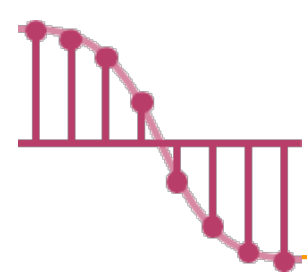
 dfn[i] = low[i] = -1;

 }

 num = 0;

}





Find the articulation points 3/3

➤ We say that u is an articulation point **iff**

"它有一個子頂點 w , 使得我們無法僅用 w , w 的後代和單一個後退邊而到達 u 的某個祖先"

✓ u is the **root of the spanning tree** and has **two or more children**

✓ or u is **not the root** of the spanning tree and u has a child w such that $low(w) \geq dfn(u)$

➤ For example,

✓ **vertex 1**: $low(0) = 4 \geq 3 = dfn(1)$

✓ **vertex 7**: $low(8) = 9 \geq 7 = dfn(7)$

✓ **vertex 5**: $low(6) = 5 \geq 5 = dfn(5)$

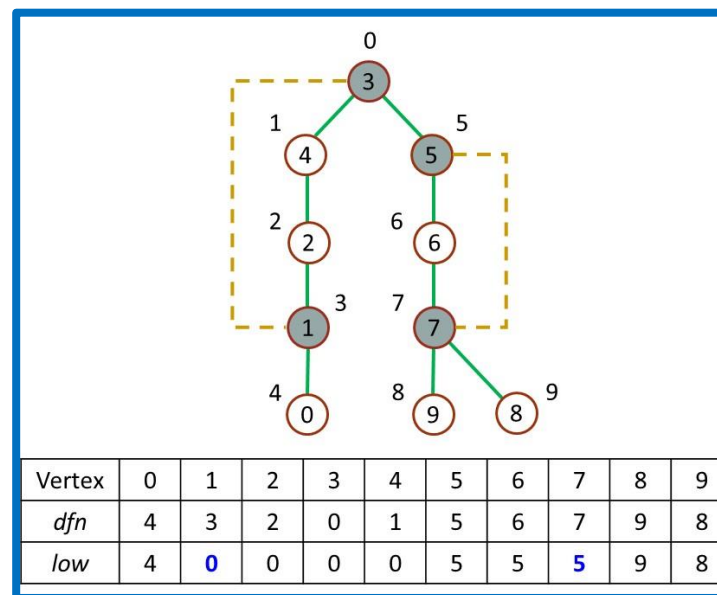
✓ **vertex 3**: it has two children

✓ **vertex 2**: $low(1) = 0 < 2 = dfn(2)$

✓ **vertex 6**: $low(7) = 5 < 6 = dfn(6)$

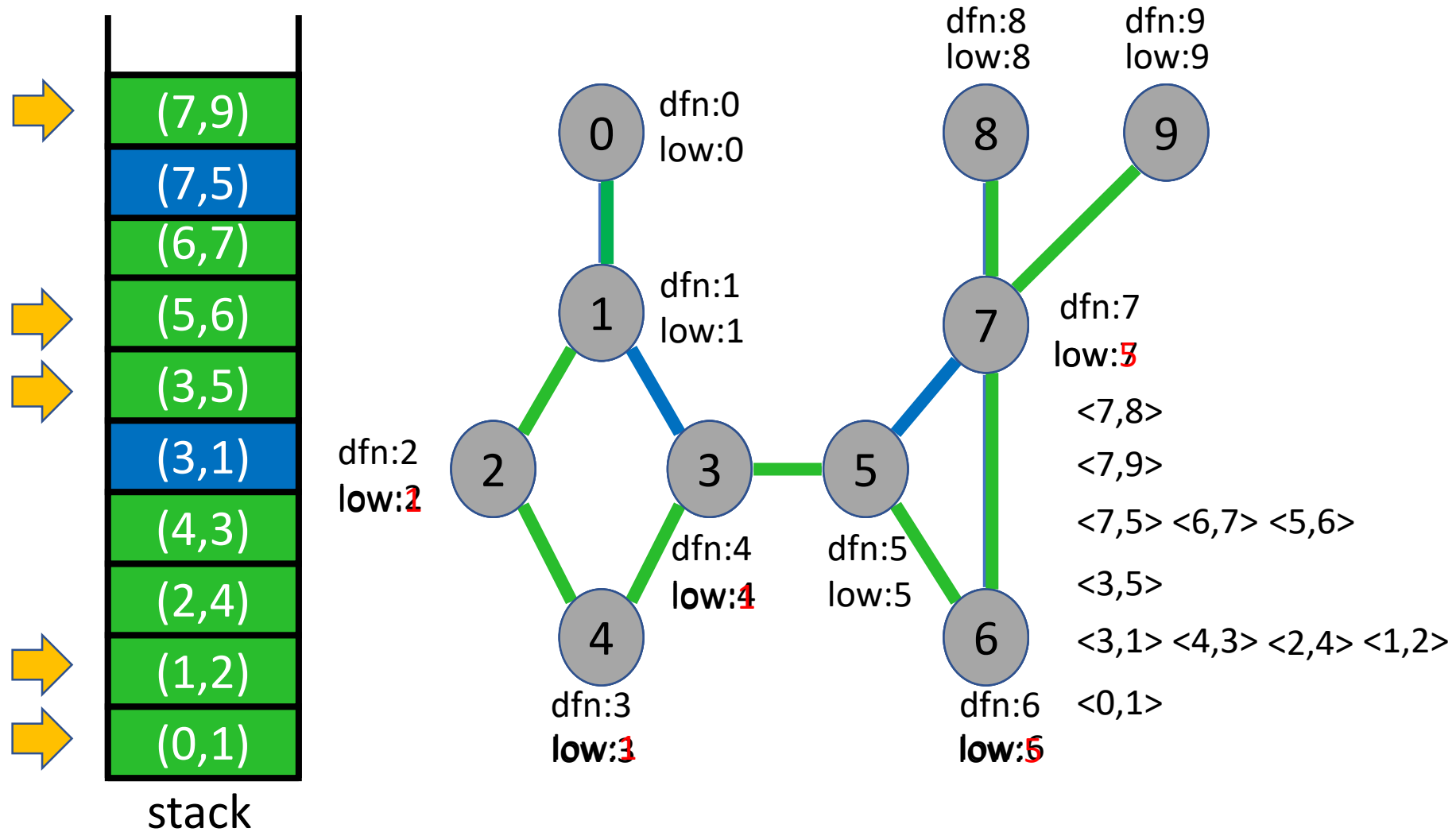
連接點

非連接點



從 $\text{dfnlow}(u,w)$ 返回時，若 $\text{low}[w] \geq \text{dfn}[u]$ ，則辨認出一個新的 biconnected component

Biconnected Graph 演算法

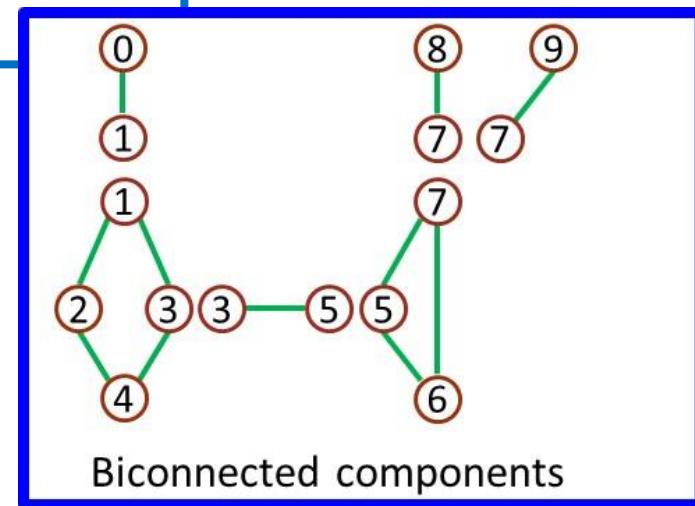
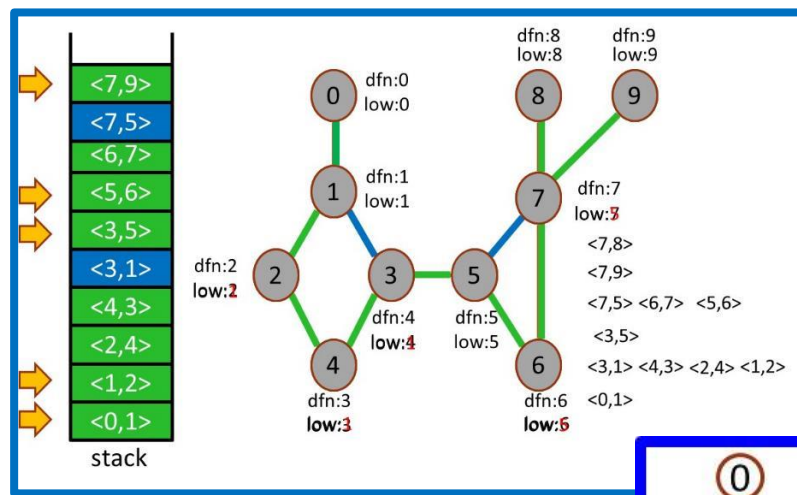


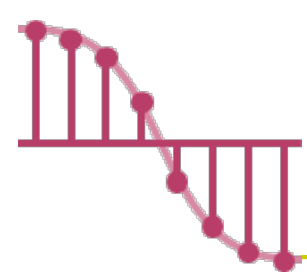
Program 6.6: Biconnected components of a graph

- Void bicon(int u, int v)
- ```
{
 /* 計算dfn與low並且依據G的雙連通元件輸出邊。v是所產生的生成樹裡u的父節點（如果v存在的話）。
 我們假設dfn[]初始值為-1，num初始值為0，s是一個初值為空的堆疊。*/
```

```
nodePointer ptr;
int w,x,y;
dfn[u] = low[u] = num++;
for (ptr = graph[u]; ptr; ptr = ptr->link) {
 w = ptr->vertex;
 if (v != w && dfn[w] < dfn[u])
 push(u,w); /* 把邊加進堆疊 */
 if (dfn[w] < 0) { /* w沒被走訪過 */
 bicon(w, u);
 low[u] = MIN2(low[u], low[w]);
 if (low[w] >= dfn[u]) {
 printf("New biconnected component: ");
 do { /* 從堆疊裡刪除一個邊 */
 pop(&x, &y);
 printf(" <%d,%d>", x,y);
 } while (!(x == u && (y == w)));
 printf("\n");
 }
 }
 else if (w != v) low[u] = MIN2(low[u], dfn[w]);
}
```

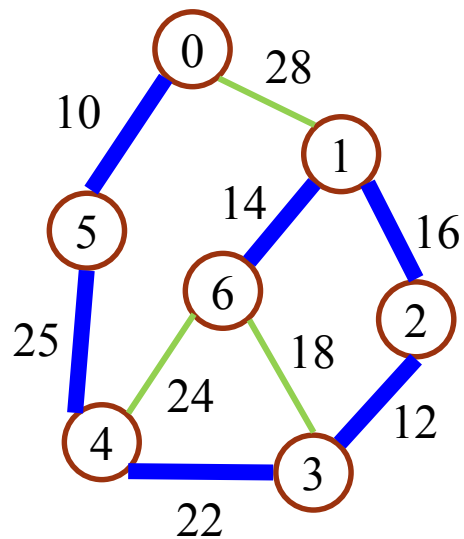
- Time Complexity:  $O(n + e)$



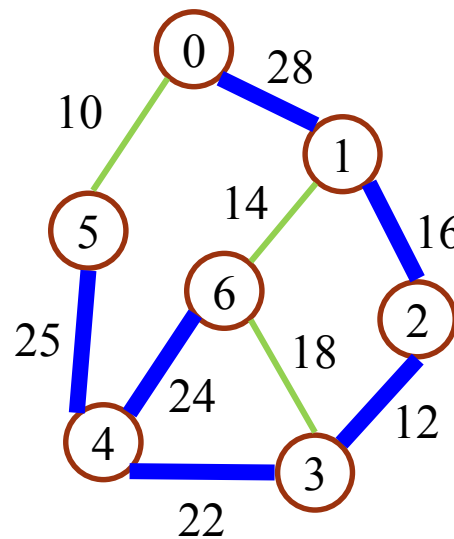


# Minimal Cost Spanning Tree <sup>1/2</sup>

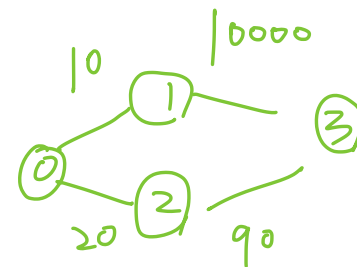
- The **cost** of a spanning tree of a weighted undirected graph is the sum of the weights of the edges in the spanning tree.
- A **minimum-cost spanning tree** is a spanning tree of least cost.

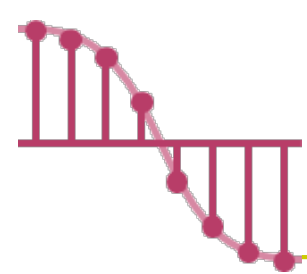


(a) minimum-cost spanning tree.  
cost=10+25+22+12+16+14=99



(b) spanning tree.  
cost=28+16+12+22+24+25=127





# Minimal Cost Spanning Tree <sub>2/2</sub>

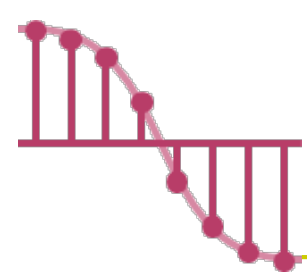
➤ Three different algorithms can be used to obtain a minimum cost spanning tree of a connected undirected graph, and all three **use greedy method**:

✓ Kruskal's algorithm

✓ Prim's algorithm

✓ Sollin's algorithm

→ 三个 algo  
都会考!

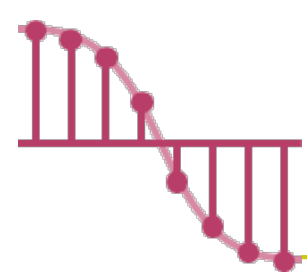


# Greedy Method <sub>1/2</sub>

---

- In the greedy method, we construct an optimal solution in stages.
- A feasible solution is one which works within the constraints specified by the problem.
- At each stage, we make a decision that is the best decision at that time.
- Typically, The selection of an item at each stage is based on either a least cost or a highest profit criterion.



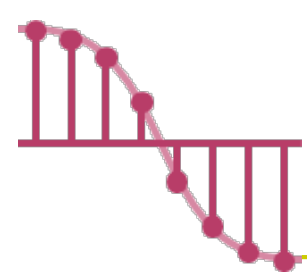


# Greedy Method <sub>2/2</sub>

➤ For minimum spanning trees, we use a least cost criterion. Our solution must satisfy the following constraints:

- ✓ We must **use only edges within the graph**
- ✓ We must **use exactly  $n-1$  edges**
- ✓ We may **not use edges that would produce a cycle**



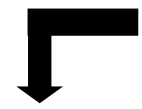
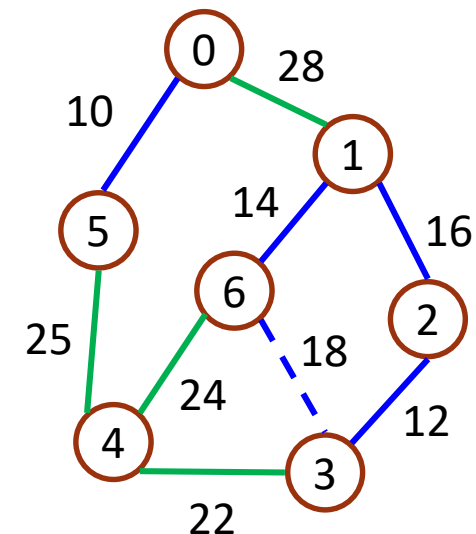
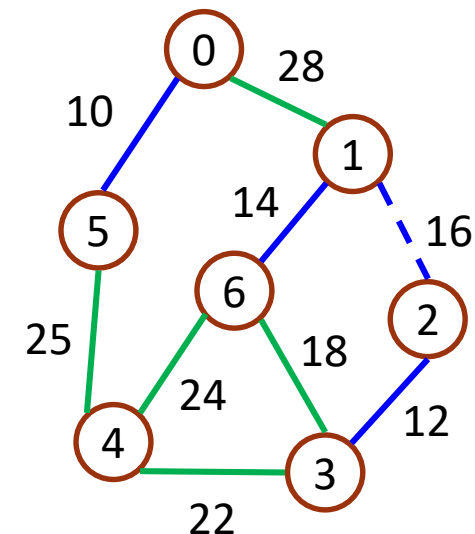
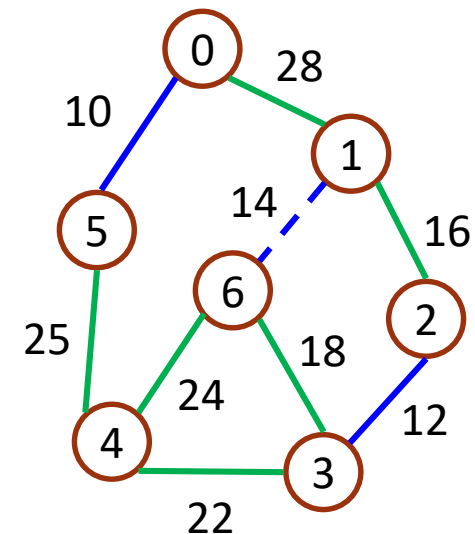
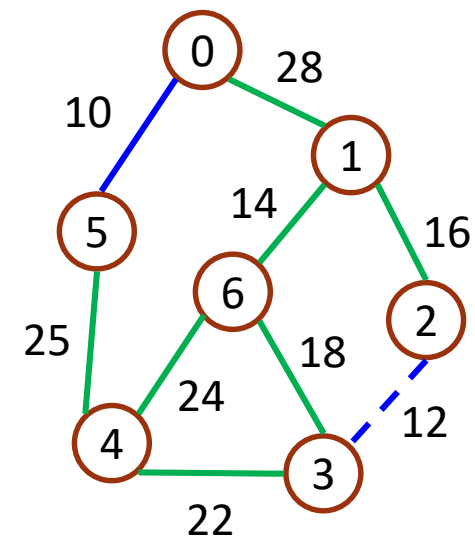
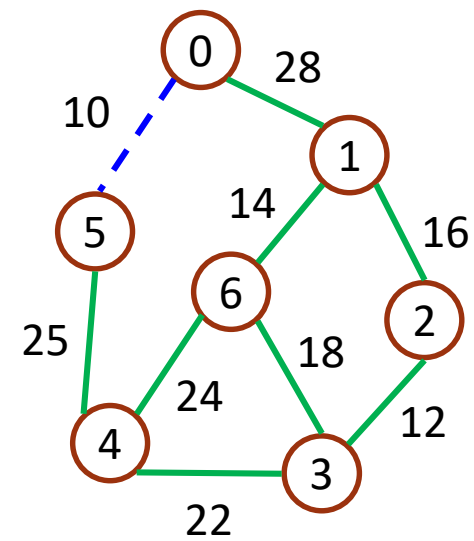
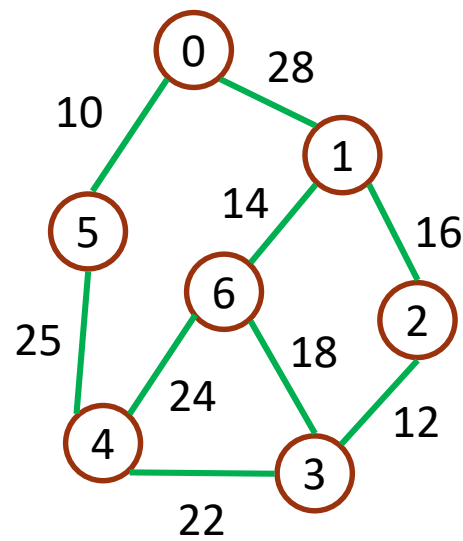
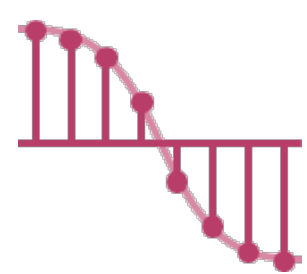


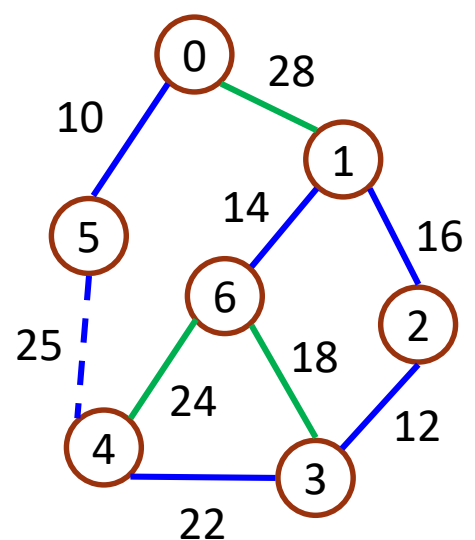
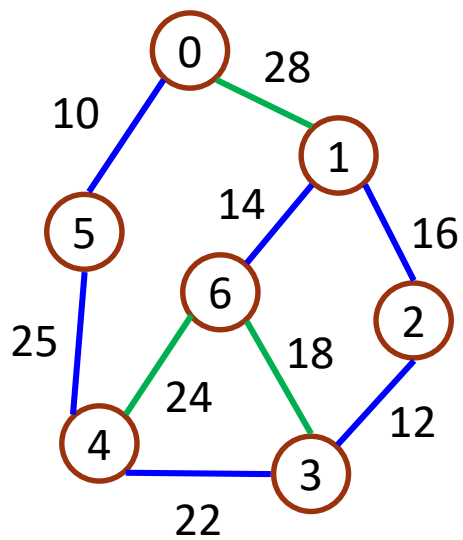
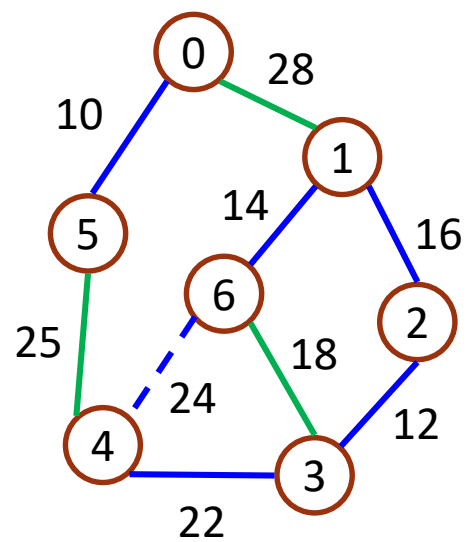
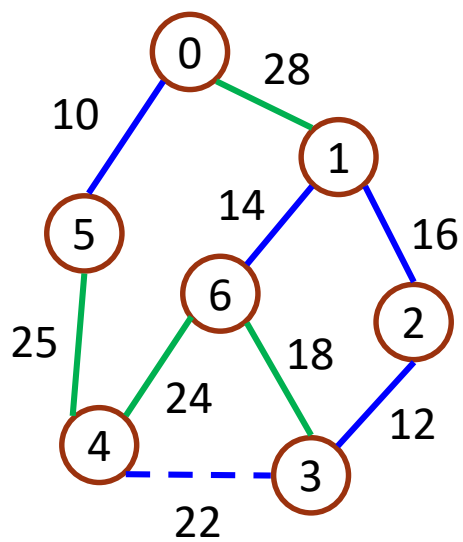
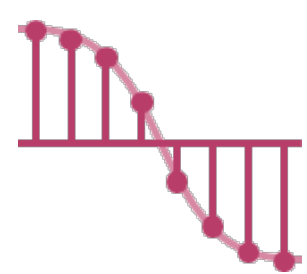
# 1. Kruskal's Algorithm <sub>1/2</sub>

会考

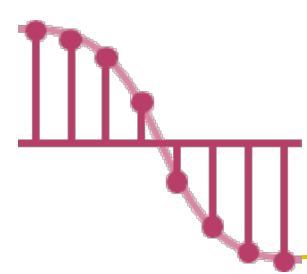
- Build a minimum cost spanning tree  $T$  by adding edges to  $T$  one at a time.
- Select the edges for inclusion in  $T$  in nondecreasing order of their cost.
- An edge is added to  $T$  if it does not form a cycle with the edges that are already in  $T$ .
- Exactly  $n-1$  edges will be selected for inclusion in  $T$ .
- **Time complexity:  $O(e \log e)$**





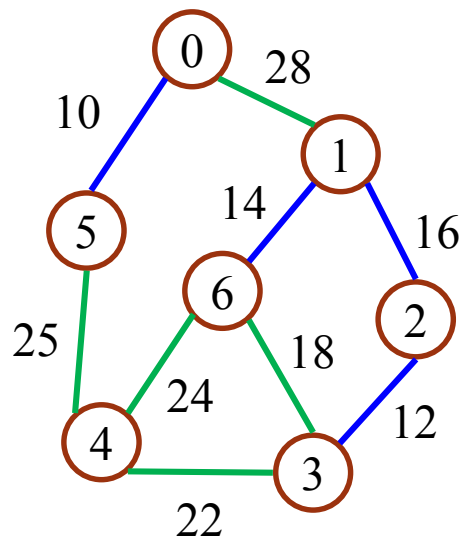


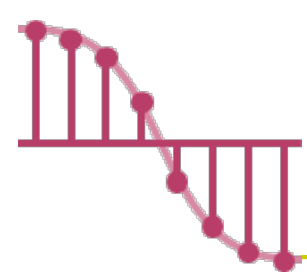




# Union-find operations

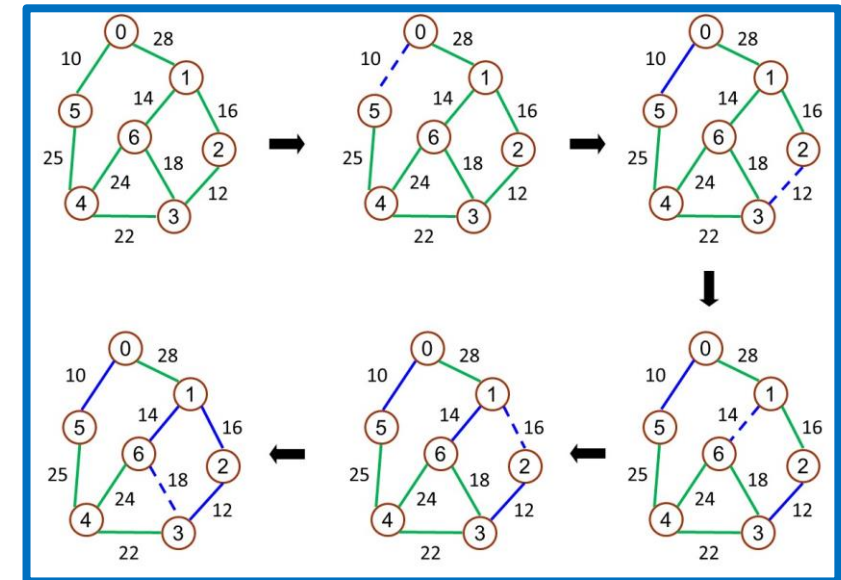
- The union-find operations are used to determine whether or not adding an edge would cause a cycle.
  - ✓ For example, the sets in the following would be  $\{0, 5\}$ ,  $\{1, 2, 3, 6\}$ ,  $\{4\}$ . (下圖的集合為  $\{0, 5\}$ ,  $\{1, 2, 3, 6\}$ ,  $\{4\}$ )
  - ✓ Since vertices 3 and 6 are already in the same set, the edge (3, 6) are rejected.





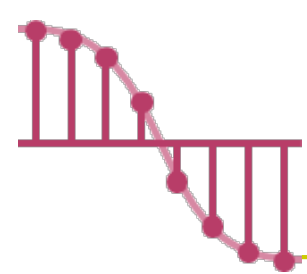
# 1. Kruskal's Algorithm 2/2

```
➤ T = { };
while (T contains less than n-1 edges && E is not empty) {
 choose a least cost edge (v,w) from E;
 delete (v,w) from E;
 if ((v,w) does not create a cycle in T)
 add (v,w) to T
 else
 discard (v,w);
}
if (T contains fewer than n-1 edges)
printf("No spanning tree\n");
```



➤ As we shall see in Chapter 7, we can sort the edges in  $E$  in  $O(e \log e)$  time.

➤ Since the union-find operations require less time than choosing and deleting an edge, the total computing time is  $O(e \log e)$ .

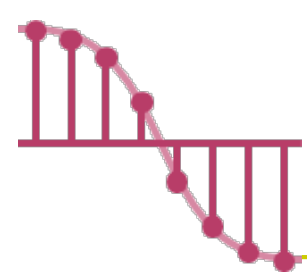


## 2. Prim's Algorithm <sub>1/3</sub>

会考!

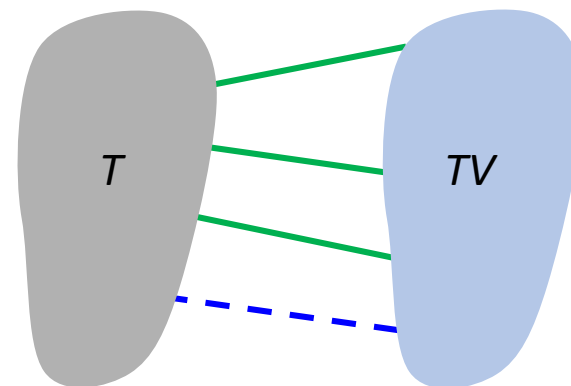
- Prim's algorithm, **like Kruskal's**, constructs the minimum-cost spanning tree one edge at a time.
- The main difference:
  - ✓ The set of selected edges **forms a tree at all times** in Prim's algorithm.
  - ✓ The set of selected edges in Kruskal's algorithm **forms a forest at each stage**.

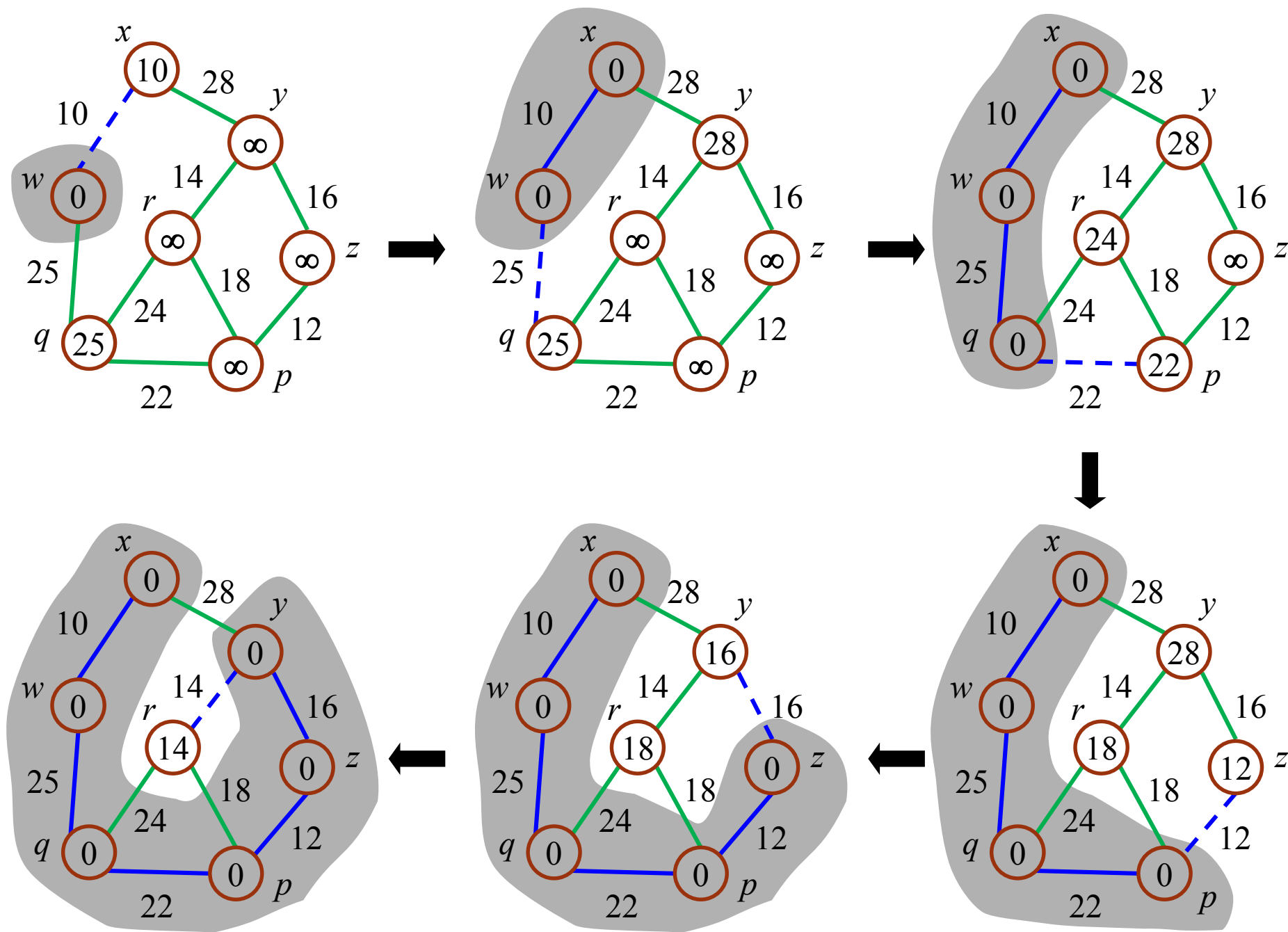
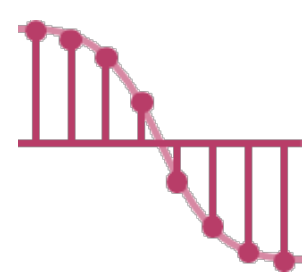


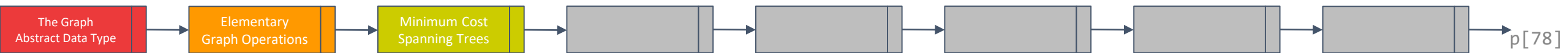
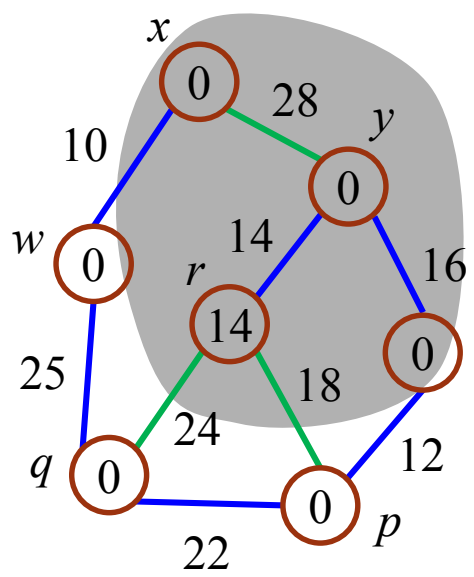
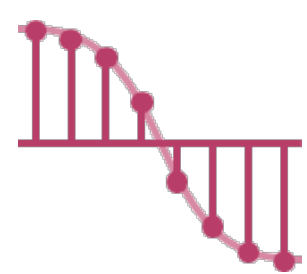


## 2. Prim's Algorithm <sub>2/3</sub>

- Prim's algorithm begins with a tree,  $T$ , that contains an arbitrary single vertex.
- Next, we add a least cost edge  $(u, v)$  to  $T$  such that  $T \cup \{(u, v)\}$  is also a tree.
- We repeat this edge addition until  $T$  contains  $n-1$  edges.
- **Time complexity:  $O(n^2)$ .**





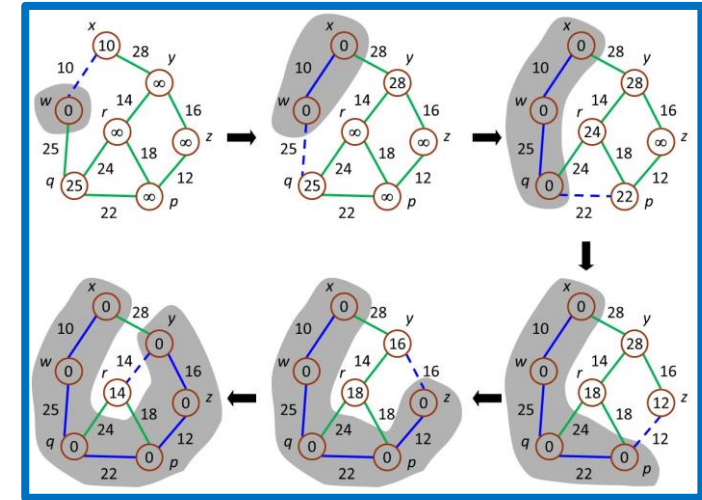


## 2. Prim's Algorithm <sub>3/3</sub>

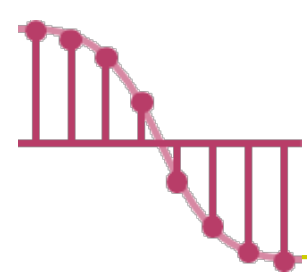
```

➤ T = { };
 TV = {0};
 while (T contains fewer than n-1 edges)
 {
 let (u, v) be a least cost edge such that u ∈ TV and v ∉ TV;
 if (there is no such edge)
 break;
 add v to TV;
 add (u, v) to T;
 }
 if (T contains fewer than n-1 edges)
 printf("No spanning tree\n");

```



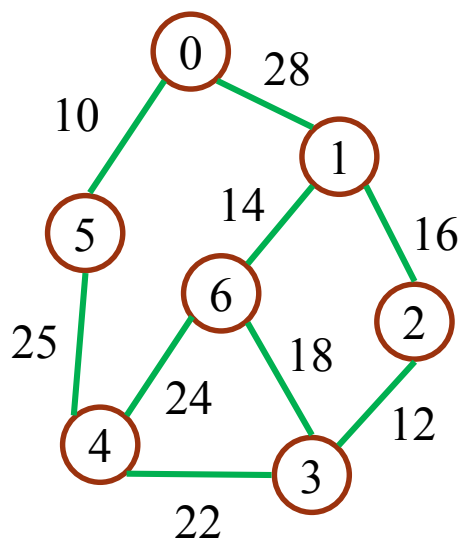
- Each vertex  $v$  that is not in  $TV$  has a companion vertex,  $near(v)$ , such that  $near(v) \in TV$  and  $cost(near(v), v)$  is minimum over all such choices for  $near(v)$ .
- Therefore, it takes  $O(n)$  time to choose an edge.
- **We can implement Prim's algorithm in  $O(n^2)$  time.**



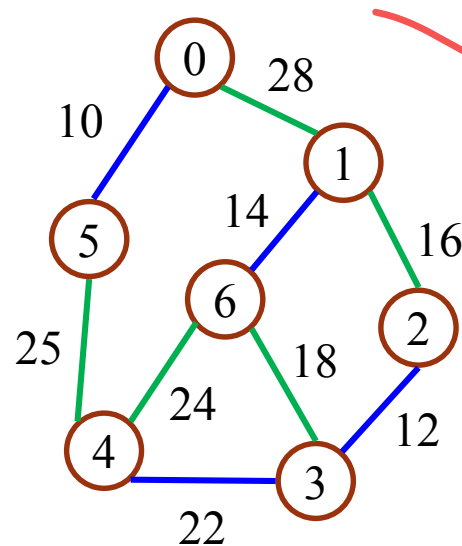
### 3. Sollin's Algorithm - An example <sub>1/2</sub>

会考!

- In stage 1, the selected edges are (0,5), (1, 6), (2, 3), (3, 2), (4, 3), (5, 0), and (6, 1).
- After eliminating the duplicate edges, we are left with edges (0,5), (1, 6), (2, 3), and (4, 3). 將所有点都視為一个獨立的連通分量!

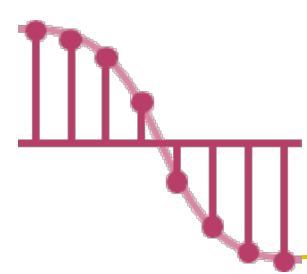


stage 1



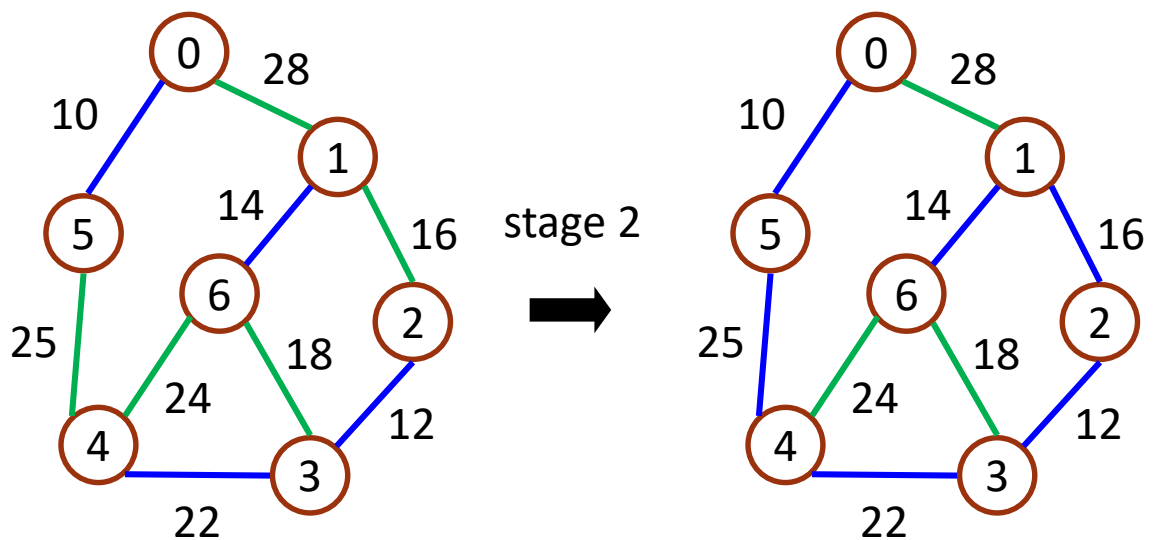
每个分量  
找最小cost的边  
直到所有分量  
連回一起

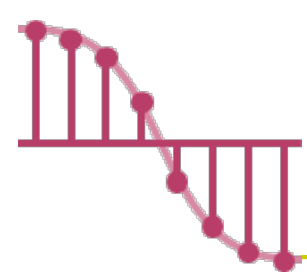




### 3. Sollin's Algorithm - An example <sub>2/2</sub>

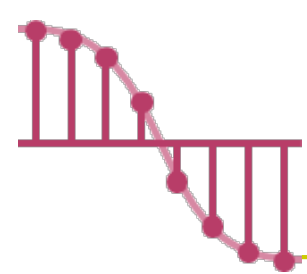
- In stage 2, the selected edges are (5,4), (1, 2), and (2, 1).
- After eliminating the duplicate edges, we are left with edges (5, 4), and (2, 1).





### 3. Sollin's Algorithm <sub>1/2</sub>

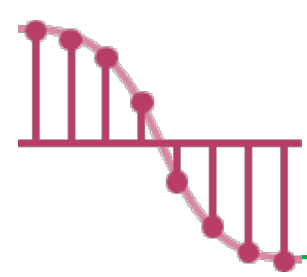
- Sollin's algorithm selects **several edges** for inclusion in  $T$  at each stage.
- At the start of a stage, the selected edges, together with all the  $n$  vertices, form a spanning forest.
- During a stage, we select one edge for each tree in the forest. This edge is a minimum cost edge that has exactly one vertex in the tree.



### 3. Sollin's Algorithm <sub>2/2</sub>

---

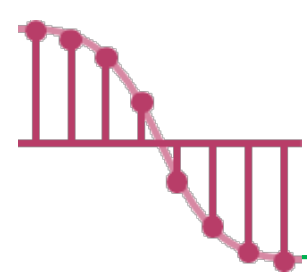
- Since two trees in the forest could select the same edge, we need to eliminate multiple copies of edges.
- At the start of the first stage the set of selected edges is empty.
- The algorithm terminates when there is only one tree at the end of a stage or no edges remain for selection.



# Shortest Paths

- How to find a shortest path from NTOU to Taipei 101 ?
  - ✓ Model this problem via a graph
  - ✓ Vertices → important locations/bus or railway stations
  - ✓ Edges → highway/railway/roads
  - ✓ Each edge has a weight representing the distance between the two stations connected by the edge.
- There are two questions:
  - ✓ **Is there a path** from NTOU to 101 ?
  - ✓ If there is more than one path, **which path is the shortest ?**
- The starting vertex of the path will be referred to as the **source** and the last vertex the **destination**.

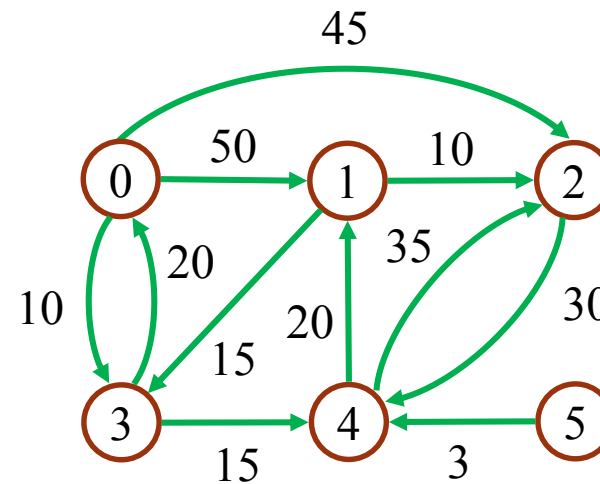


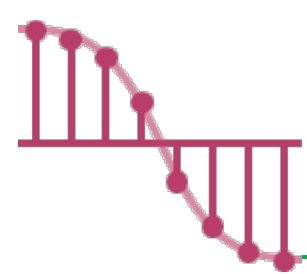


# Single Source/All Destinations: Nonnegative Edge Costs

- We are given a directed graph,  $G=(V, E)$ , a weighting function,  $w(e)$ ,  $w(e)>0$ , for the edges of  $G$ , and a source vertex  $v_0$ . If  $v_i$  and  $v_j$  are not adjacent, then  $e(v_i, v_j) = \infty$ .
- We wish to determine a shortest path from  $v_0$  to each of the remaining vertices of  $G$ .

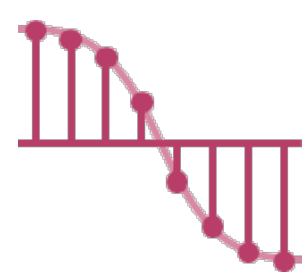
|   | <b>Path</b> | <b>Length</b> |
|---|-------------|---------------|
| 1 | 0, 3        | 10            |
| 2 | 0, 3, 4     | 25            |
| 3 | 0, 3, 4, 1  | 45            |
| 4 | 0, 2        | 45            |





# Dijkstra's Algorithm

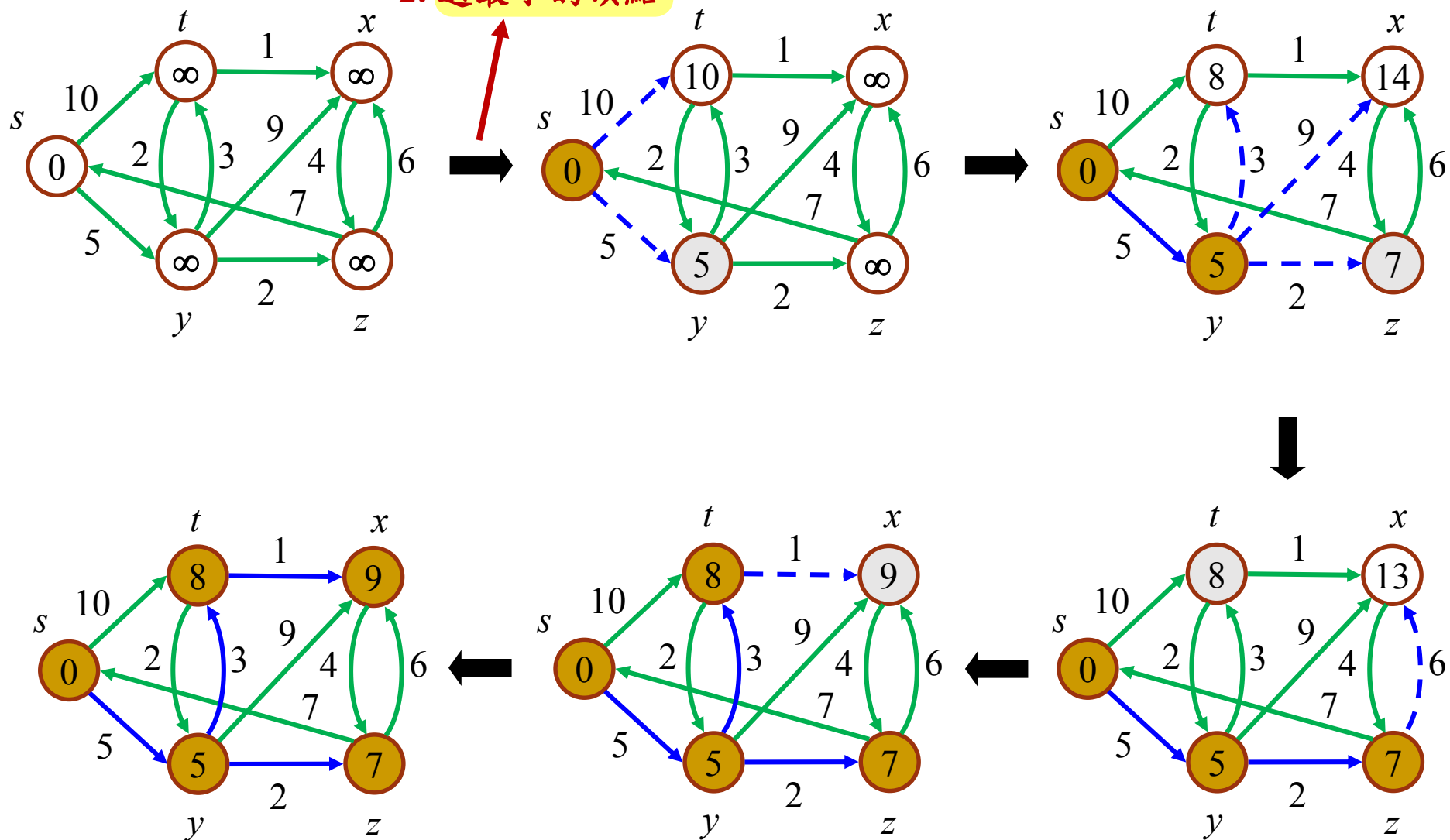
- We may use a greedy algorithm to generate the shortest paths in the order indicated in the above example.
- Let  $S$  denote the set of vertices, including  $v_0$ , whose shortest paths have been found.
- For  $v$  not in  $S$ , let  $dist[v]$  be the length of the shortest path starting from  $v_0$ , going through vertices only in  $S$ , and ending in  $v$ .

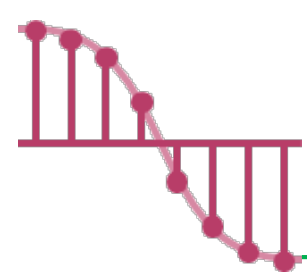


每回合都做兩件事：

1. 更新剩餘的頂點值

2. 選最小的頂點

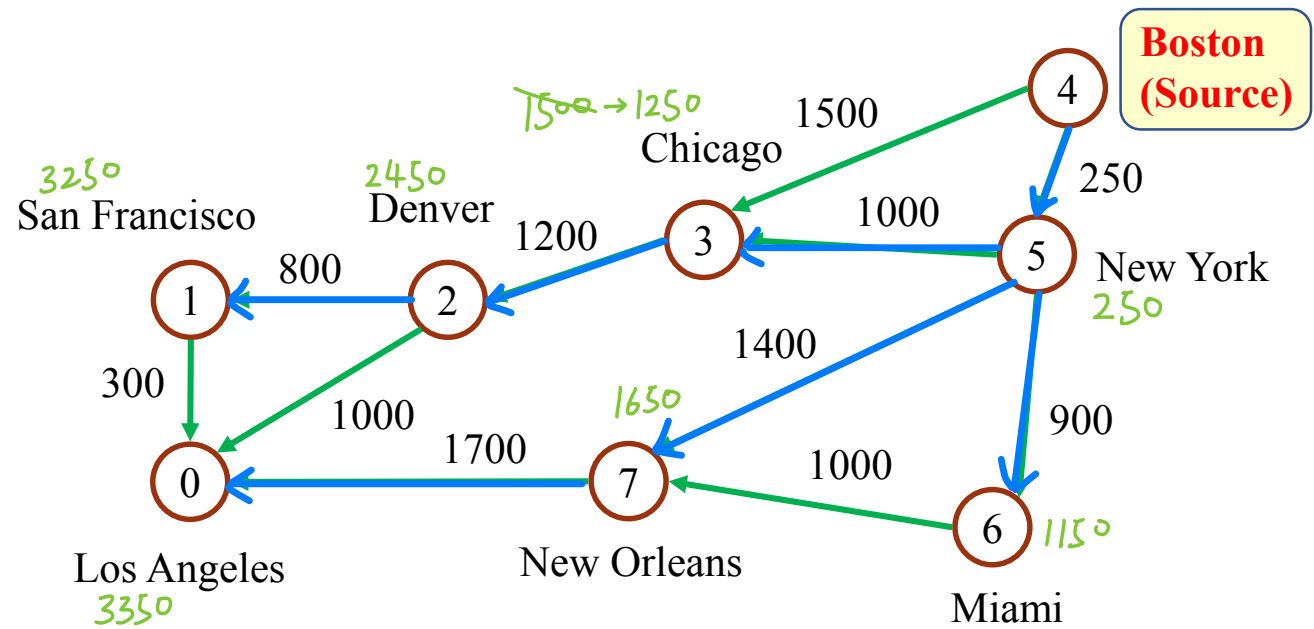
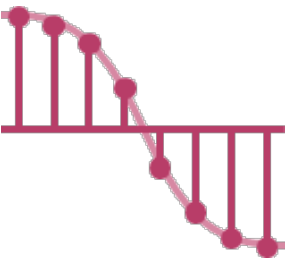




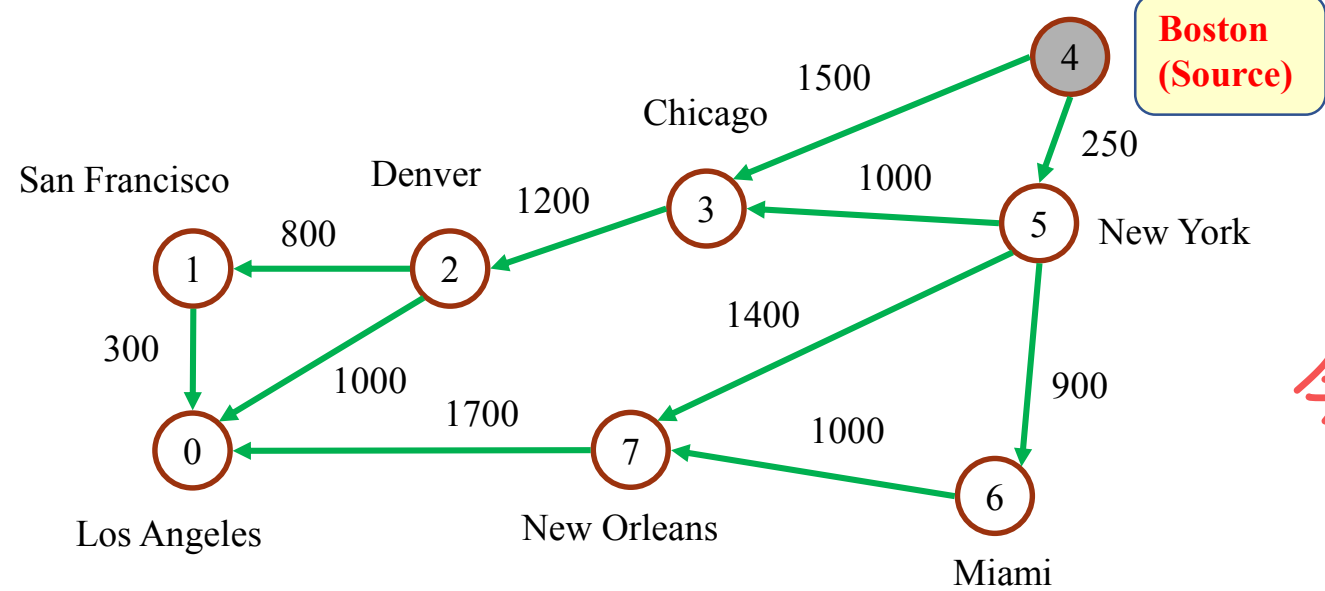
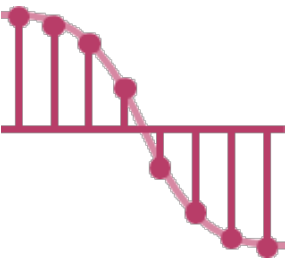
# Dijkstra's Algorithm

- At the first stage, we add  $v_0$  to  $S$ , set  $dist[v_0] = 0$  and determine  $dist[v]$  for **each**  $v$  not in  $S$ .
- Next, at each stage, vertex  $w$  is chosen so that it has the minimum distance,  $dist[w]$ , among all the vertices not in  $S$ .
- Adding  $w$  to  $S$ , and updating  $dist[v]$  for  $v$ , that is not currently in  $S$ .
- We repeat this vertex addition until  $S = V(G)$ .
- **Time complexity:  $O(n^2)$ .**



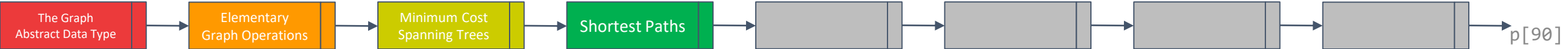


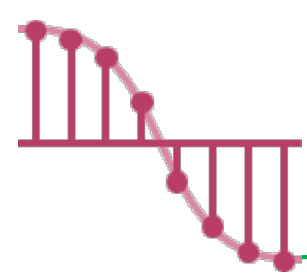
|   | 0    | 1   | 2    | 3    | 4 | 5   | 6   | 7    |
|---|------|-----|------|------|---|-----|-----|------|
| 0 | 0    |     |      |      |   |     |     |      |
| 1 | 300  | 0   |      |      |   |     |     |      |
| 2 | 1000 | 800 | 0    |      |   |     |     |      |
| 3 |      |     | 1200 | 0    |   |     |     |      |
| 4 |      |     |      | 1500 | 0 | 250 |     |      |
| 5 |      |     |      | 1000 |   | 0   | 900 | 1400 |
| 6 |      |     |      |      |   |     | 0   | 1000 |
| 7 | 1700 |     |      |      |   |     |     | 0    |



会考!

| Iteration | Vertex Selected | Distance |          |          |      |      |     |          |          |
|-----------|-----------------|----------|----------|----------|------|------|-----|----------|----------|
|           |                 | LA       | SF       | DEN      | CHI  | BOST | NY  | MIA      | NO       |
|           |                 | [0]      | [1]      | [2]      | [3]  | [4]  | [5] | [6]      | [7]      |
| Initial   | ---             | $\infty$ | $\infty$ | $\infty$ | 1500 | 0    | 250 | $\infty$ | $\infty$ |
| 1         | 5               | $\infty$ | $\infty$ | $\infty$ | 1250 | 0    | 250 | 1150     | 1650     |
| 2         | 6               | $\infty$ | $\infty$ | $\infty$ | 1250 | 0    | 250 | 1150     | 1650     |
| 3         | 3               | $\infty$ | $\infty$ | 2450     | 1250 | 0    | 250 | 1150     | 1650     |
| 4         | 7               | 3350     | $\infty$ | 2450     | 1250 | 0    | 250 | 1150     | 1650     |
| 5         | 2               | 3350     | 3250     | 2450     | 1250 | 0    | 250 | 1150     | 1650     |
| 6         | 1               | 3350     | 3250     | 2450     | 1250 | 0    | 250 | 1150     | 1650     |
| 7         | 0               | 3350     | 3250     | 2450     | 1250 | 0    | 250 | 1150     | 1650     |





# Dijkstra's Algorithm

➤  $S \leftarrow \{v_0\};$

$dist[v_0] \leftarrow 0;$

**for each  $v$  in  $V - \{v_0\}$  do  $dist[v] \leftarrow e(v_0, v);$**

while  $S \neq V$  do

**choose a vertex  $w$  in  $V-S$  such that  $dist[w]$  is a minimum;**

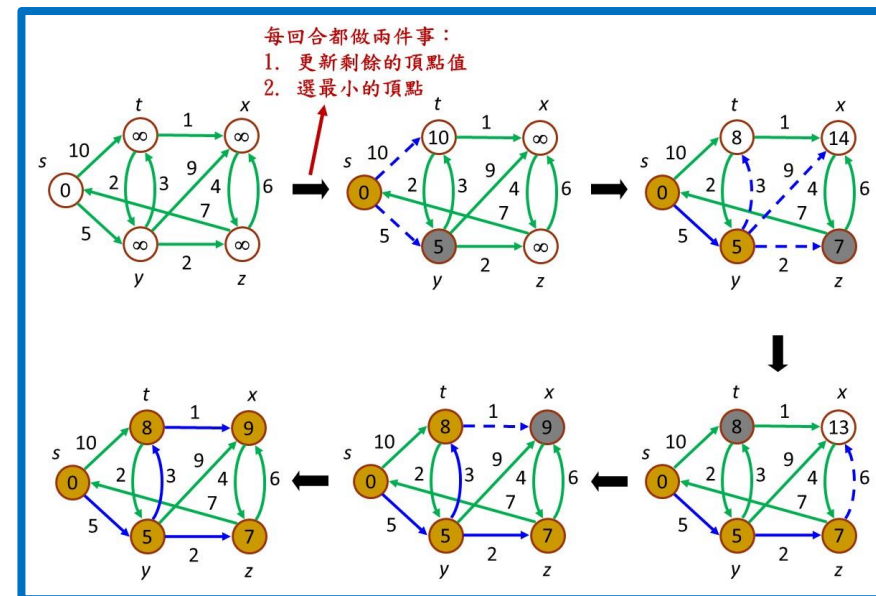
add  $w$  to  $S$ ;

for each  $v$  in  $V-S$  do

**$dist[v] \leftarrow \min(dist[v], dist[w] + e(w, v));$**

endfor

endwhile



# Program 6.9 & 10:

## Single Source Shortest Paths – Dijkstra Algorithm <sup>1/2</sup>

➤ **void shortestpath (int v, int cost[][MAX\_VERTICES], int distance [], int n, short int found [])**

{

*/\* distance [i] represents the shortest path from vertex v  
to i, found[i] holds a 0 if the shortest path from vertex i  
has not been found and a 1 if it has, cost is the adjacency  
matrix \*/*

int i, u, w;

for (i=0; i<n; i++) {

    found [i] =FALSE;

    distance [i] =cost[v][i];

}

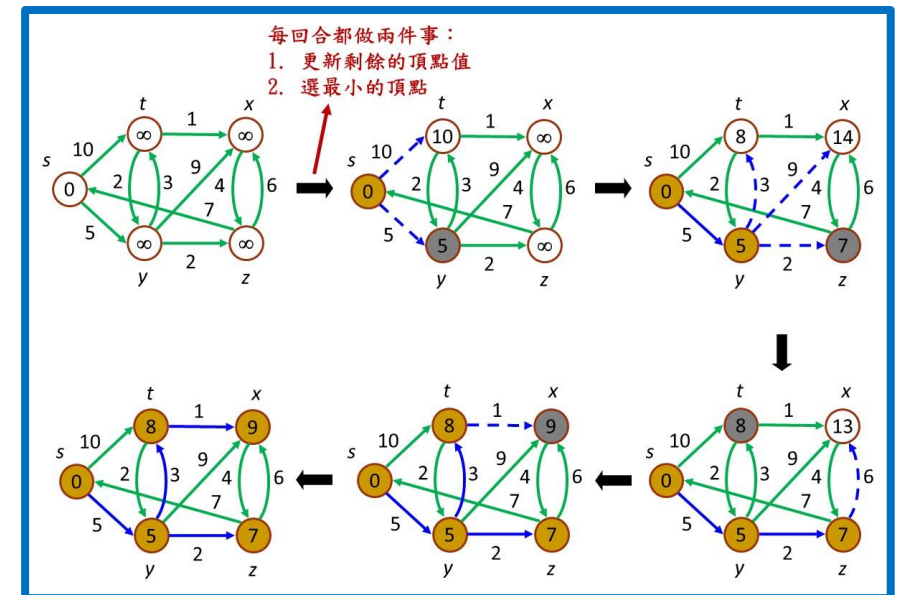
found[v] =TRUE;

distance[v] = 0;

for (i=0; i<n-1; i++) {

**u= choose (distance, n, found);**

    found[u] =TRUE;



# Program 6.9 & 10:

## Single Source Shortest Paths – Dijkstra Algorithm <sub>2/2</sub>

```

for (w=0; w<n; w++)
 if(!found[w])
 if (distance[u] + cost[u][w] < distance[w])
 distance[w] = distance[u]+cost[u][w];
 }
}

```

```

int choose (int distance[], int n, short int found[])
{

```

```

 /* find the smallest distance not yet checked */

```

```

 int i, min, minpos;

```

```

 min = INT_MAX;

```

```

 minpos = -1;

```

```

 for (i=0; i<n; i++)

```

```

 if (distance[i] < min && !found[i]){

```

```

 min = distance[i];

```

```

 minpos = i;

```

```

 }

```

```

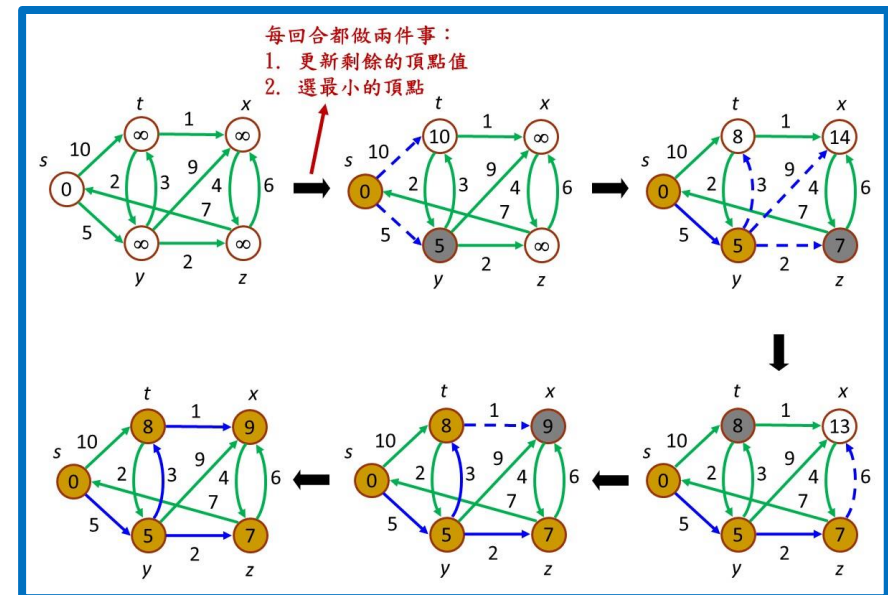
 return minpos;

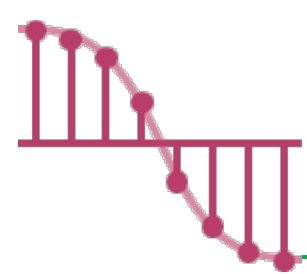
```

```

}

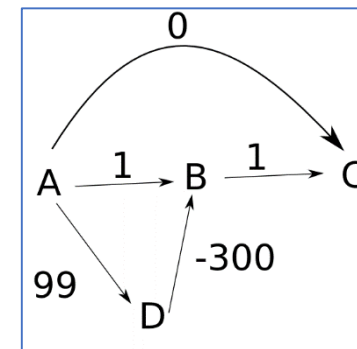
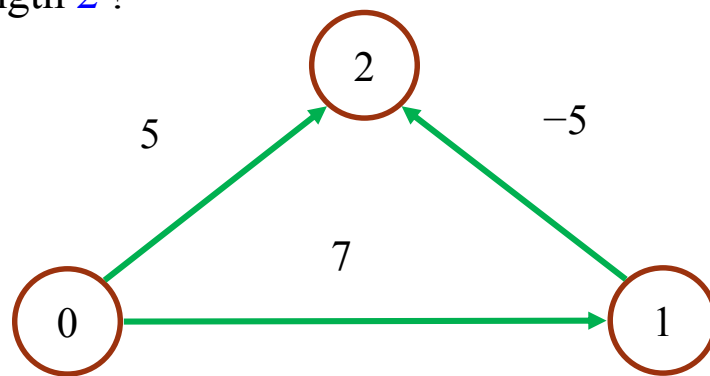
```



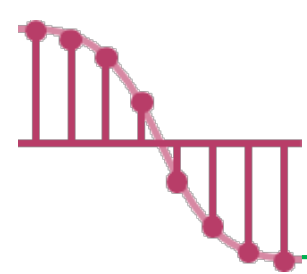


# Single Source/All Destinations: General Weights

- We now consider the general case when some or all of the edges of the directed graph  $G$  may have negative length.
- The function `shortestpath` does **not necessarily work**.
  - ✓ According to the function, we have  $dist[1]=7$ ,  $dist[2]=5$ .
  - ✓ But the shortest path from 0 to 2 is 0, 1, 2.
  - ✓ This path has length **2** !

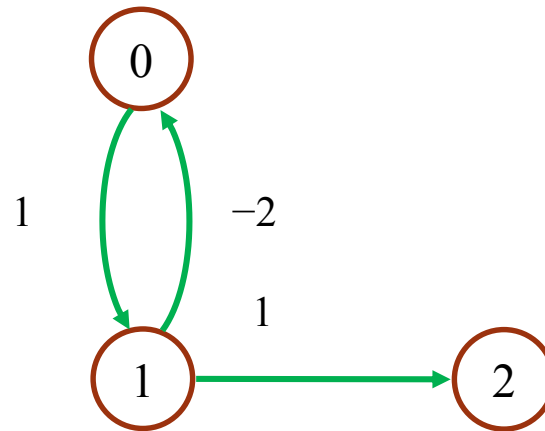


| Iteration | Vertex Selected | Distance |      |   |    |
|-----------|-----------------|----------|------|---|----|
|           |                 | A        | B    | C | D  |
| Initial   | ---             | 0        | 1    | 0 | 99 |
| 1         | C               | 0        | 1    | 0 | 99 |
| 2         | B               | 0        | 1    | 0 | 99 |
| 3         | D               | 0        | -201 | 0 | 99 |

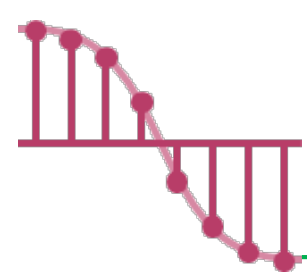


# No negative cycle is permitted !

- When negative edge lengths are permitted, we require that the graph have no cycles of negative length.
- This is necessary so as to ensure that shortest paths consist of a finite number of edges.

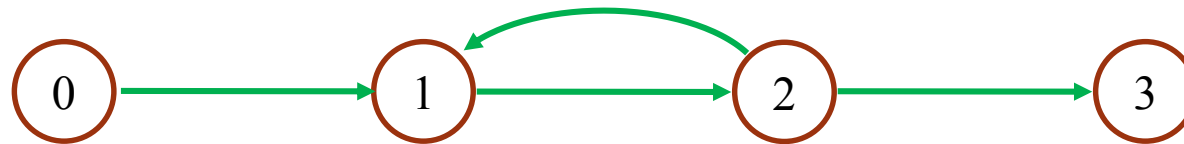


Directed graph with a cycle of negative length.

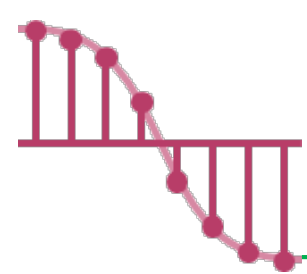


# Observation

- When there are no cycles of negative length, there is a shortest path between any two vertices of an  $n$ -vertex graph that has at most  $n-1$  edges on it.
  - ✓ Otherwise, the path must repeat at least one vertex and hence must contain a cycle.
- So, elimination the cycles from the path results in another path with the same source and destination. The length of the new path is no more than that of the original.







# Definition

---

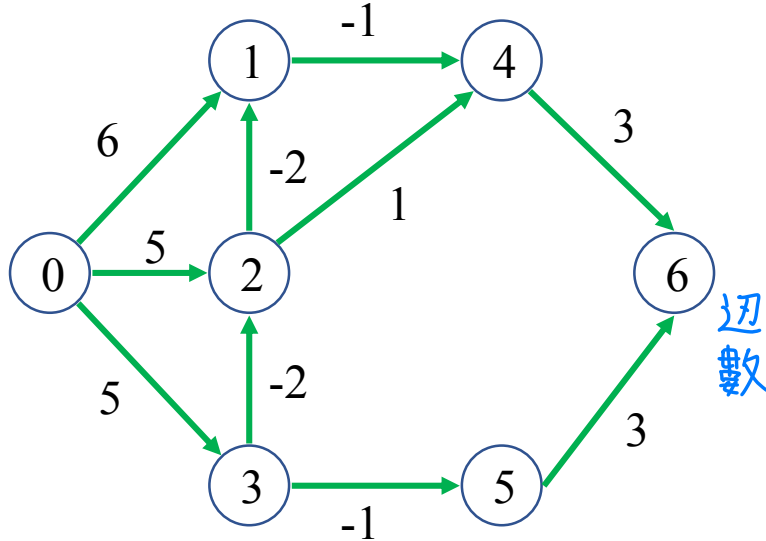
- Let  $\text{dist}^k[u]$  be the length of a shortest path from the source vertex  $v$  to vertex  $u$  under the constraint that the shortest path contains at most  $k$  edges.
- Then,  $\text{dist}^1[u] = \text{length}[v][u]$ ,  $0 \leq u < n$ .
- Our goal then is to compute  $\text{dist}^{n-1}[u]$  for all  $u$ .
- This can be done using the **dynamic programming** methodology.

# Figure 6.31:

## Shortest paths with negative edge lengths

$$dist^k[u] = \min\{dist^{k-1}[u], \min\{dist^{k-1}[i] + length[i][u]\}\}$$

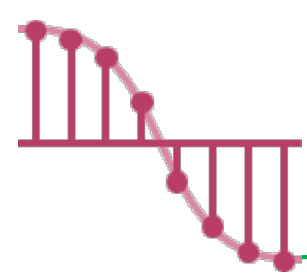
会考!



| $k$ | $dist^k[u]$ 点 |   |   |   |          |          |          |
|-----|---------------|---|---|---|----------|----------|----------|
|     | 0             | 1 | 2 | 3 | 4        | 5        | 6        |
| 1   | 0             | 6 | 5 | 5 | $\infty$ | $\infty$ | $\infty$ |
| 2   | 0             | 3 | 3 | 5 | 5        | 4        | $\infty$ |
| 3   | 0             | 1 | 3 | 5 | 2        | 4        | 7        |
| 4   | 0             | 1 | 3 | 5 | 0        | 4        | 5        |
| 5   | 0             | 1 | 3 | 5 | 0        | 4        | 3        |
| 6   | 0             | 1 | 3 | 5 | 0        | 4        | 3        |

(a) A directed graph

(b)  $dist^k$



# The Main Spirit of Bellman-Ford Algorithm

- If the shortest path from  $v$  to  $u$  with at most  $k$ ,  $k > 1$ , edges has no more than  $k-1$  edges, then  $dist^k[u] = dist^{k-1}[u]$ .
- If the shortest path from  $v$  to  $u$  with at most  $k$ ,  $k > 1$ , edges has exactly  $k$  edges, there exists a vertex  $i$  such that  $dist^{k-1}[i] + length[i][u]$  is minimum.
- These observations result in the following recurrence for  $dist$ :

$$dist^k[u] = \min\{dist^{k-1}[u], \min\{dist^{k-1}[i] + length[i][u]\}\}$$